BITTENSOR SUBNET · WHITEPAPER

RamCove

More AI. Same GPU.

A Bittensor subnet that helps every GPU serve more AI requests, by rewarding verified improvements in memory efficiency and useful serving capacity.

> We help GPUs do more: verified software that serves more qualified AI requests on the same GPUs. Customers request a profile with no upfront fee, and terms are agreed only after an improvement is shown.

Subnet name	RamCove
Version	Whitepaper v0.2
Date	September 2026
Audience	GPU and serving engineers; miners and validators; customers; investors

02 Abstract and executive explanation

What RamCove does. RamCove is a Bittensor subnet with one job: serve more AI requests on the same GPU. It pays for software improvements, verified by the subnet's operator-run validator rather than by their authors, that let one fixed GPU serve more useful language-model work by using its memory better. Miners submit reviewable code changes to a pinned open-source serving engine. The operator-run validator builds that code in controlled environments, tests it against conversations generated only after the code is frozen, and measures *qualified serving capacity*: the highest sustained request rate at which answers stay correct, fast enough and error-free. Every other validator sets its weights from the operator's signed results, without running these tests itself. Winning improvements are merged into an open reference stack and turned into tested deployments for customers who want them. Each competition runs on a published *profile* (one open model, one engine commit, one GPU type), and profiles are added over time across model families and GPUs (Section 6.4); the first is Qwen3.8-27B on the NVIDIA H100 SXM, which this paper's examples use.

The customer problem. A platform team serves an open-weight model on 80 GB GPUs. Each GPU holds the model's weights plus a *KV cache*, the working memory kept for every conversation in progress. In this paper's illustrative example the cache budget is 40 GiB and one long conversation takes about 2 GiB of it, so about nineteen fit. When the cache is full, the server evicts conversations and recomputes them later, or makes new requests wait. RamCove adds verified software that fits more qualified work into the same memory.

How the competition works. Submissions arrive continuously. Submitting is free: a free check rejects malformed or duplicate entries, and when the evaluation queue has room a submission from a registered hotkey is admitted at no charge, with the operator providing the GPU time. The artifact is frozen, briefly screened, then evaluated in order of admission. Within each published profile there is one **champion**, King #1 of a queue of five kings (*king of the hill*). Each king below King #1 earns 20% of the profile's miner allocation, and King #1 earns the rest, including the share of every empty slot: a lone King #1 earns 100%, and with five kings each earns 20%. A challenger that beats King #1 becomes the new King #1, every king moves down one place, and King #5 leaves the queue. The champion is measured once, when it is crowned, and every challenger is evaluated once against that fixed score. A challenger replaces the champion only when it serves more than 5% more qualified requests (a result close to that margin gets one more reading; the margin is calibrated to measured noise before submissions open), and its quality, response-time, resource and memory-evidence checks all pass. Ties and unclear results keep the incumbent. Until a challenger beats the tuned baseline there is no king, and the miner allocation is burned rather than given to anyone. Every validator sets its weights from the operator's signed result, which the operator serves only to validators.

What customers pay for. Customers submit the model, hardware and workload they want improved. RamCove funds the initial work on its own infrastructure: the team assesses the request, tunes the customer's configuration on its own GPUs as the baseline to beat, and develops improvements at its own cost while the subnet's miners compete on the published profile. After a useful improvement is verified and demonstrated, RamCove and the customer negotiate payment, delivery and any further services. Released code remains available under its license. There is no upfront research or profile fee, though a customer's own staff time and testing are real costs. Prices are negotiated with each customer and are not fixed in this paper. Customer money never passes through the chain; miners are paid in the subnet's alpha token by emissions and pay nothing to be evaluated.

How RamCove grows. RamCove's software covers the whole lifecycle: rules engine, free checks, admission, the ordered queue, evaluation workers, promotion review, weight plans and a public dashboard. The team launches with its own resources. Launch needs a working evaluator on real H100 hosts, a tested starting baseline published as the version-0 reference, and the security and chain-readiness checks; it does not need an improvement to have been found. Customer profile requests, team-funded development and the miner competition, a verified improvement, a customer demonstration and a negotiated agreement follow. Customer payments are RamCove's commercial revenue. Evaluating miner submissions is the team's operating cost, reported separately.

> **HOW TO READ THIS PAPER**
>
> Numbered citations point to sources listed at the end with access dates; Appendix D records each claim's evidence type (documentation, inspected code, chain read, or estimate). Every threshold is a default that each profile's calibration sets from measured noise and freezes before submissions open, and every number in an example is illustrative and labelled.

Contents

02 Abstract and executive explanation
03 The problem and why now
04 Product and customer workflow
05 How a submission is judged, end to end
06 Technical approach, profiles and hardware
07 Network architecture, submission lifecycle and queue
08 Evaluation: what is measured and how
09 Champion rule and incentives
10 Security, auditability and trust assumptions
11 Why RamCove is built this way
12 Commercial model, costs and funding
13 Why Bittensor, token mechanics and governance
14 Launch, operation and risks
Appendix A · Profile, admission, weight-result and report formats
Appendix B · Workload, statistics, parameters, queue and costs
Appendix C · Pseudocode
Appendix D · Claim and evidence ledger
Appendix E · Open policy and commercial decisions
Appendix F · Glossary
References

03 The problem and why now

3.1 Memory is the quiet limit on serving

A language model reads the whole prompt, then writes the answer one token at a time, each token looking back at everything before it. To avoid re-reading the prompt, the server keeps a **KV cache**: the key and value vectors every attention layer computed for each token. It lives in GPU memory, grows with context length, and is kept separately for each conversation.

It is large even in efficient models. The first profile's model, Qwen3.8-27B, keeps full attention in 16 of its 64 layers, which store 64 KiB per token at 16-bit precision, and its other 48 layers keep a fixed recurrent state of about 75 MiB per conversation [1, 2]. A 32,768-token conversation therefore holds about 2.07 GiB, and in the example of Section 4.6 about nineteen of them fill a 40 GiB budget. Engines allocate this memory in blocks to avoid waste [3], but cannot create more. When it runs out, vLLM refuses new requests or preempts running ones and recomputes them later [4]. Either way, users wait.

Memory binds for document assistants for three reasons. Document conversations routinely carry 8,000 to 32,000 tokens, well within the model's 262,144-token native context [5]. Decoding is limited by memory bandwidth, and the batch that amortizes it is limited by how many caches fit. And users pause between turns: keeping their caches, and for a hybrid model their recurrent state, resident avoids recomputation [6, 7], but only while memory lasts. A smaller cache helps only if it keeps the information answers need and does not slow every step with compression work.

3.2 What already exists

The first profile's engine, vLLM, already supports 8-bit cache storage, prefix caching, chunked prefill, and host-memory offloading [8, 9]; its maintainers report, as an author-reported result, that 8-bit storage cut cache memory to about 54% with small quality losses [10]. Research adds eviction, low-bit quantization, query-aware sparsity, and head-level retention [11, 12, 13, 14, 15, 16]. Nearly all results are author-reported, and careful studies document failures: eviction loses facts that later questions need, and sub-linear-memory methods degrade in multi-turn use [17, 18, 19, 20]. What is missing is a standing, adversarial test of which ideas survive real service limits, and a way to pay for turning them into maintained code.

3.3 Why speed benchmarks are not enough

Request speed is easy to improve in ways customers do not pay for: shorter answers, too few concurrent requests for memory to bind, warmed caches, or a queue left to grow during a short run. A contest that rewards speed alone invites exactly these shortcuts. The industry's reference benchmark instead fixes latency limits and asks how much Poisson-arriving load a system carries while 99% of requests meet them [21]. Serving research calls this goodput: the highest request rate that keeps service-level objectives (SLOs, promised response-time limits) met for a target share of requests [22]. Load must be open-loop, because a generator that waits for responses throttles itself exactly when the server is overloaded [23].

3.4 Why now

Permissively licensed open models run production assistants on one GPU [5]; open engines accept patches [24, 25]; load tools report SLO attainment and goodput [26, 27]; a public randomness beacon, drand, exists and Bittensor already uses its timelock [28, 29]. Together these make a continuous, measured competition for serving capacity practical now.

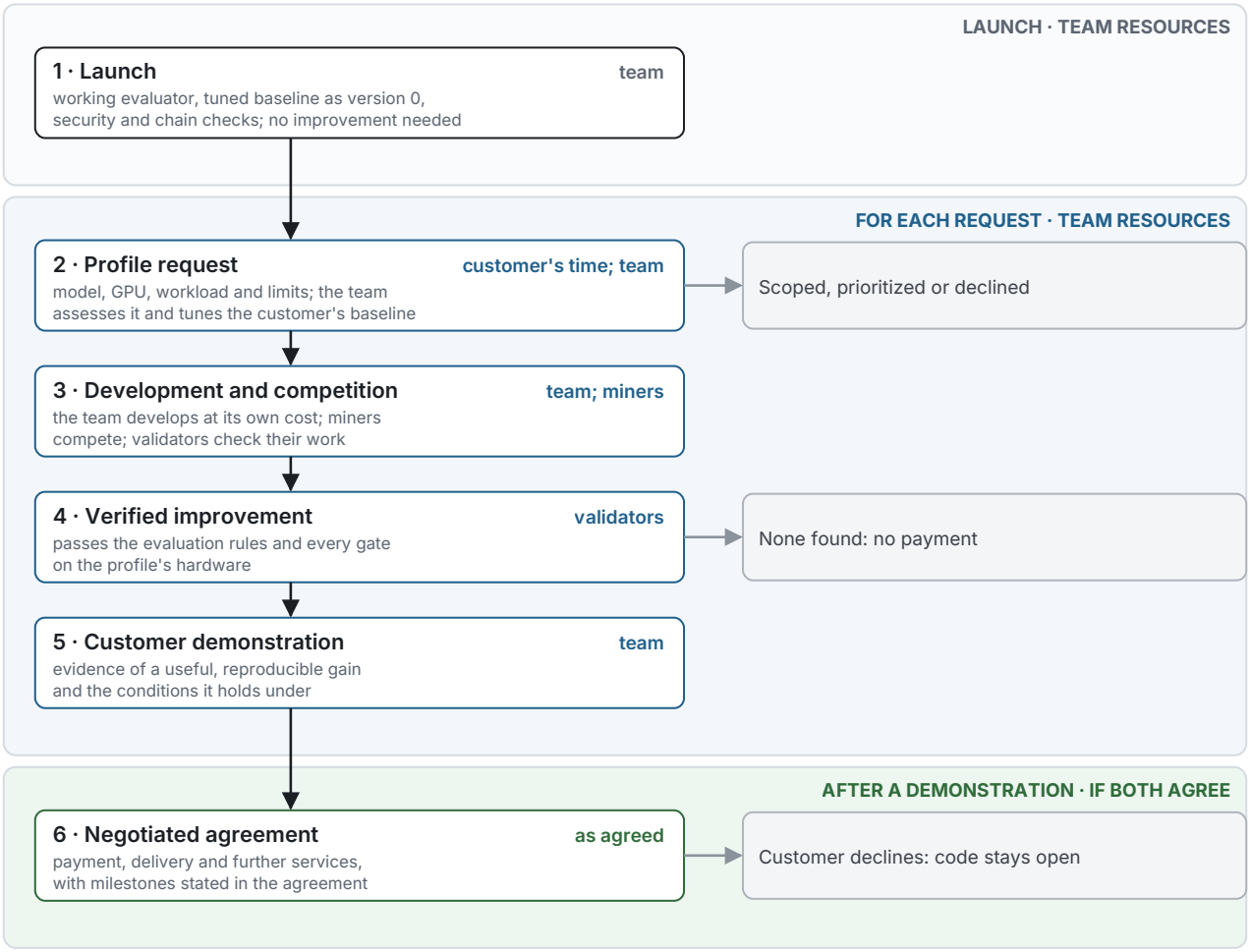
04 Product and customer workflow

4.1 What RamCove offers

Customers submit the model, hardware and workload they want improved. RamCove funds the initial work on its own infrastructure. After a useful improvement is demonstrated, the team and customer agree on commercial terms for the result and any further services. Released code remains available under its license.

The work always runs in this order:

1. **Launch.** Using its own resources, the team launches the subnet with a working evaluator on real H100 hardware, a tested starting baseline and the necessary security and chain-readiness checks (Section 14). Each profile opens with its **tuned baseline published as the version-0 reference**. Launch does not require an improvement to have been discovered already.
2. **A profile request.** A customer describes the configuration it wants improved: model, GPU type, workload, quality requirements and response-time requirements. The team decides whether it can support the request and, if so, tunes the customer's configuration on team-controlled hardware. That tuned baseline, not an untuned default, is what any improvement must beat.
3. **Team-funded development and the miner competition.** The team develops and tests improvements at its own cost. The profile is published, the subnet's miners compete to improve it, and the operator-run validator checks their work (Sections 5 and 8).
4. **A verified improvement.** An improvement passes the evaluation rules and every gate on the profile's hardware (Sections 8 and 9).
5. **A customer demonstration.** The customer receives evidence of the useful, reproducible improvement and the conditions under which it works: the hardware, limits, quality results and configuration it was measured on.
6. **A negotiated agreement, if both sides want one.** RamCove and the customer negotiate payment, delivery and any further services after the demonstration. Payment and delivery milestones are stated in their agreement, and further work follows the agreed scope.



No fee to RamCove before an agreement. The customer's own costs are its staff time and any testing it runs.

Figure 1. From launch to an agreement. The subnet launches with a tested baseline, not with an improvement already found. The team carries the cost of launch, assessment, the tuned baseline, its own development and the demonstration. The customer carries its own effort: describing its workload, and any testing it chooses to run. Commercial terms are negotiated only after a verified improvement has been demonstrated, and the customer may decline.

The customer never holds a token or touches the chain. The *operator* is the operating company that runs the competition's rules and delivers customer work.

Profile 1 exists before any request. Qwen3.8-27B served by vLLM on the H100 SXM is a profile the team defined to exercise the protocol on a widely deployed model and GPU, and it is the profile the subnet launches with. No customer requested it.

4.2 Who pays for what

Stage	Who pays	What it costs
Launch, assessment, tuned baseline, development, demonstration	The team, from its own resources	GPU time, engineering, evaluation, security review, operation (Section 12.3)
Submitting a profile request	Nobody: there is no upfront research or profile fee	The customer's own time to describe its workload and share statistics
Customer-side testing, if the customer wants to reproduce a result on its own hardware	The customer, for its own hardware and staff time	Unknown until the configuration is known
The accepted improvement, integration, deployment and further work	Agreed after the demonstration, if the customer wants it	Set in the agreement; nothing is fixed in advance

A request does not commit the team to unlimited work or guarantee an improvement. Requests can be scoped, prioritized or declined according to the team's resources. After a demonstration, RamCove and the customer may negotiate payment for the improvement and for the delivery or further services the customer wants, or the customer may decline. A customer's payment is RamCove's commercial revenue for demonstrated

improvements and agreed services, not an investment in RamCove. "No upfront fee to RamCove" does not mean the customer has no costs: its staff time, any testing on its own hardware, and its own deployment work are real, and they are stated when they are known.

4.3 Two different measurements

A miner beating the champion means that a submission served more qualified requests than the current champion on the profile's evaluation hardware, by the published margin, with every gate passed (Section 9.2). That decides the subnet's reward.

A customer receiving useful value means that an improvement serves more of *that customer's* work than its own tuned baseline, under its own quality and response-time requirements, on its configuration. That decides whether a commercial agreement makes sense.

The two are related but not the same. A competition win does not automatically meet a customer's purchasing requirements: the customer's traffic, limits and purchasing arrangements may differ from the profile's. The demonstration in step 5 is what shows customer value.

Team work is reference work. Each profile's version-0 reference is its tuned baseline. Improvements the team develops later are measured under the same evaluation rules and published, attributed to the team, as reference work: they enter the profile only as a baseline update announced in advance as a new profile version (Section 9.4). They earn no miner reward. Operator-affiliated hotkeys cannot be champions (Section 9.3), so miners are paid only for beating the current reference by the margin.

4.4 Why a customer pays for open-source work

Released improvements are open source. A customer can use released code under its license whether or not it signs an agreement, no subscription is needed to keep an existing open-source build running, and other users may later run the same code. What a customer pays for is the work around the code:

- **Finding an improvement for its own configuration.** The search targets its model, hardware and workload, not a generic benchmark.
- **Evidence.** Measurements under its quality and response-time requirements, with the conditions under which the improvement holds.
- **Adaptation and delivery.** Integrating the improvement into its deployment, with a staged rollout and a way back.
- **Further work it chooses to buy.** New improvements, re-testing when its engine or model changes, and support, under whatever scope it agrees.

Open-source usage rights come from the license. Obligations such as delivery dates, support and acceptance come only from a signed agreement. RamCove does not claim exclusivity, secrecy, lock-in, automatic renewal or guaranteed retention. A customer can install published improvements itself, its own engineers can tune further, and upstream engines keep improving. RamCove is worth paying only while it finds, validates and delivers useful improvements faster or more reliably than those alternatives (Section 11.3).

4.5 Who the early customers are likely to be

A **platform team already serving an open-weight model on its own or rented GPUs**, with conversational, document-heavy traffic and a capacity ceiling set by how many GPUs it runs: an inference provider, a software company embedding an assistant, or an enterprise running one internally. A customer's *GPU fleet* is simply its collection of GPUs. There is no minimum fleet size. Whether a request suits RamCove depends on the value that can be measured and on the team's ability to support it.

4.6 The illustrative example used throughout

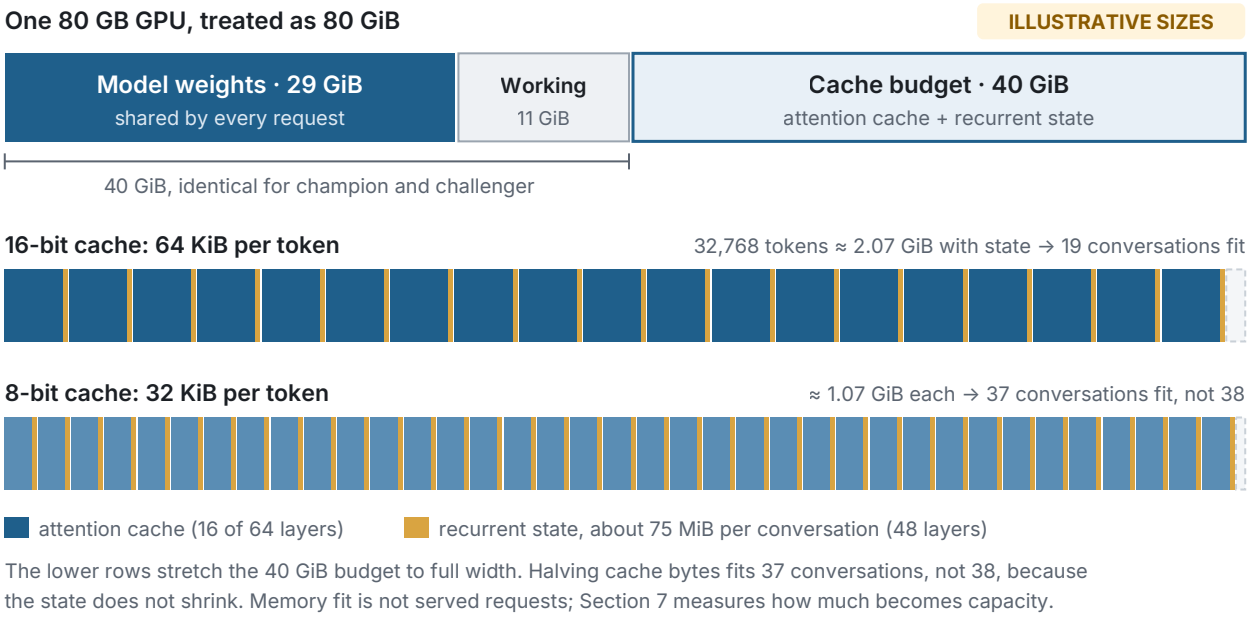


Figure 2. Where the memory goes on one GPU (illustrative sizes). Weights are shared by every request and fixed for the profile; working memory covers activations, buffers, captured graphs, and a driver margin; the cache budget holds every resident conversation's attention cache and recurrent state. The lower rows compare 16-bit and 8-bit cache entries at the same budget. Round numbers in binary units, sized to resemble the first profile.

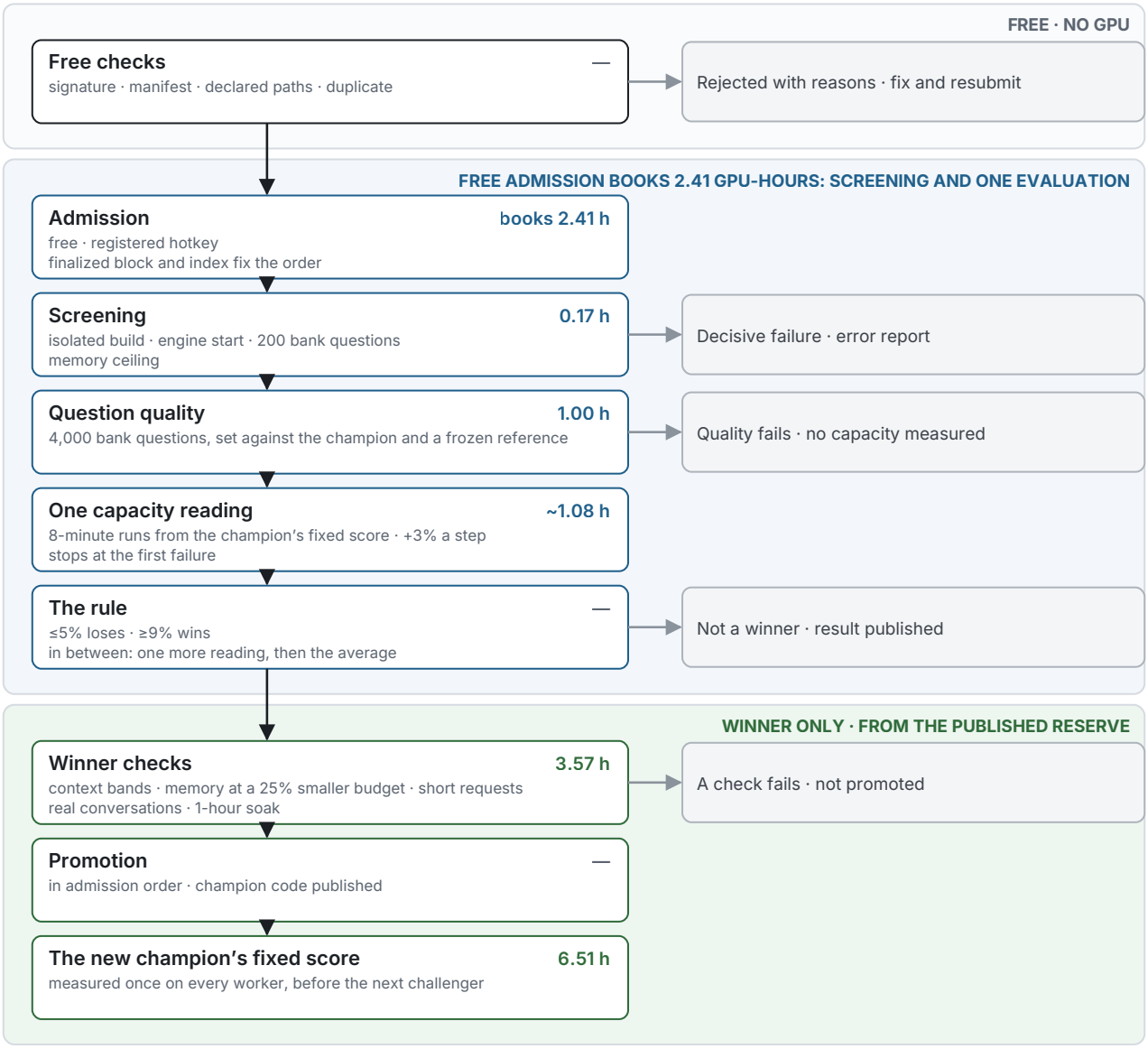
ILLUSTRATIVE EXAMPLE · USED IN SECTIONS 3, 4 AND 8

A team serves one model replica per 80 GB GPU, treated as 80 GiB for round numbers (a real H100 reports about 79.6 GiB). Weights, working memory, and a driver margin take **40 GiB**, leaving **40 GiB** for cache. A 32,768-token conversation needs 2 GiB of attention cache (64 KiB per token) plus about 75 MiB of recurrent state, so **nineteen** fit; with 8-bit cache entries it needs about 1.07 GiB and **thirty-seven** fit, not thirty-eight, because the recurrent state does not shrink. That is memory fit only. Whether it becomes more served requests depends on decode speed within the response-time limits and on whether answers stay correct. Section 8's illustrative load test shows a 25% capacity gain, not a doubling.

Code production and inference hosting are separate: miners are paid from emissions for verified code; customers run their own inference and pay the operator only under an agreement. No customer request ever reaches a miner.

05 How a submission is judged, end to end

This chapter is the whole path a submission takes, in one place: what happens at each step, why that step exists, and what it costs. Later chapters give the definitions and the exact thresholds — the lifecycle and queue in Section 7, the measurements in Section 8, the replacement rule in Section 9.2 — but nothing in them changes the shape described here.



A miner pays once. Screening plus one evaluation is what the queue is capped against; the reserve pays for a winner.

Figure 3. What happens to one submission. Free checks cost nothing, and so does a submission. Admission books screening plus one evaluation of the operator's GPU time, and that booking is what the queue is capped against. Quality is judged before capacity, so a stack that answers worse never reaches a load test. The reading climbs a published grid from the champion's fixed score and stops at the first run that fails. Only a winner reaches the extra checks, which run on the published reserve rather than on the submission's booking. Numbers are planning estimates.

5.1 The problem this workflow has to solve

Two things are true at once. A capacity measurement on a shared, busy GPU is noisy, so any single run can mislead. And every evaluation costs real GPU time, which the operator provides, so the design cannot simply repeat everything until the noise disappears.

The workflow answers both by spending its GPU time where it changes the decision:

- **Cheap checks first.** Signature, manifest and patch scope cost nothing and run before a submission joins the queue. Screening costs ten GPU-minutes and rejects work that does not build, does not start or answers far worse than the champion.
- **Quality before capacity.** A stack that serves more requests by answering them worse is not an improvement. Quality is checked on questions the miner has never seen, and failing it ends the evaluation before a single load run.
- **One comparison, against a number already known.** The champion is measured once per worker, when it is crowned, and its score is then fixed. A challenger is measured once against that number. Measuring both sides for every submission would double the bill and add the champion's noise to the challenger's.

- **The expensive checks only for a winner.** Memory evidence, the short-request control, real conversations and the hour-long soak together cost more than an evaluation. They run only after a winning reading, and they use the published reserve.

5.2 The steps

Free checks. The artifact's signature, manifest, declared paths, licences, size limits and lineage are checked with no GPU. A miner can fix and resubmit at once.

A place in the queue, free. Submitting costs nothing. When the queue has room and the miner's hotkey is registered on the subnet, the submission is admitted and **2.41 GPU-hours** of the operator's evaluation time are booked for it: screening plus one evaluation. When the queue is full, it joins the free waiting list and is invited as capacity frees. The finalized block at admission, and the submission's place among that block's admissions, fix the order in which submissions are tested and promoted; finishing early never overtakes an earlier admission.

Screening (about 10 GPU-minutes). An isolated build, an engine start within five minutes, 200 questions from the current operator bank and the memory ceiling at low load. Speed is not judged here and no winner is crowned here. A build failure, a start failure, a ceiling breach or smoke accuracy more than 15 points below the champion's ends the submission.

Question quality (about 1 GPU-hour). The challenger answers the period's whole bank — 4,000 questions it has never seen — and its answers are compared question by question with the champion's answers to the same questions and with a frozen reference implementation (Section 8.6). The bank is built by the operator platform, not by validators: a secret seed per seven-day period, its fingerprint anchored a day ahead and the seed published once the period ends, so anyone can rebuild the questions and check the grading afterwards. Failing quality ends the evaluation, and no capacity is measured.

One capacity reading (about 1.1 GPU-hours). The reading starts at the champion's fixed score and steps up 3% after every eight-minute run that passes the per-run gates, up to 60% above that score. The reading is the highest rate that passed, and it stops at the first rate that fails. Three runs is the expected length.

The rule. The gain is the reading divided by the champion's fixed score, minus one. At or below 5% the challenger loses; at or above 9% it wins; in between it gets exactly one more reading and the average of the two decides (Section 9.2). Because the comparison is arithmetic against a fixed number, a winner overtaken by a newer champion can be re-compared without measuring anything again.

Winner checks (about 3.6 GPU-hours, from the reserve). Each check answers something a single reading cannot: attainment in every context band, that the gain survives a 25% smaller cache budget and shows up in the accounting of resident conversations, that short single-turn requests did not get worse, that the stack holds up when each turn is built from its own previous answer, and that an hour at 90% of the measured capacity shows no leak or drift.

Promotion. Promotion happens in admission order. The new champion's patches are published, and every worker measures the new champion's fixed score again before the next challenger is judged against it.

5.3 What it costs, and who provides it

> **GPU TIME, PLANNING ESTIMATES**
>
> **Booked for each admitted submission:** screening 0.17 + one evaluation 2.24 = **2.41 GPU-hours**. That is the number the queue cap counts. The operator provides this time; the miner pays nothing.
> **Run on the reserve (20% of each worker's week):** a winner's extra checks, about 3.57 GPU-hours, and a new champion's fixed score, about 6.51 GPU-hours per worker. Neither is booked against a submission.

A submission is free whatever its result. One stopped by an operator-side failure runs again at the same admission key, within the published retry limits.

5.4 What is visible while it runs

Every state change is written to a signed event log, and the public dashboard shows where each admitted submission stands: the queue in admission order, what is on a GPU right now and how far through its step it is (screening questions answered, bank questions answered, which run of which reading and at what rate, which winner check), and why each rejected submission was rejected. The rules, the profile, the bank fingerprints and the queue depth are public before a miner submits anything.

06 Technical approach, profiles and hardware

6.1 Submissions and candidate methods

A submission is a source-only patch or plug-in against the profile's exact engine commit, restricted to declared paths, with a signed manifest (Appendix A). One artifact must serve every request in its profile. The families below are the main directions. Their published results are author-reported, which is why Section 8 tests every submission independently.

Family	Why it might raise capacity	Risks the gates must catch
Cache quantization to 8, 4, or fewer bits [13, 14, 30]	Fewer bytes per token and per step	Quality loss; decompression that breaks per-token limits
Selective retention and eviction [11, 12, 31]	More conversations resident	Losing a fact a later question needs [20, 18, 32]
Head- and layer-level policies [16]	Full history only where needed	Misclassified heads; model-specific tuning
Memory-aware admission, batching, retention	Fewer recomputations; higher prefix hits	Starving some users; caught by per-band attainment
Cache layout and allocation [33]	More usable bytes from the same budget	Repair overhead; correctness bugs
Efficient compressed-cache kernels [15, 34]	Removes the decode penalty of compression	Kernel correctness
Capped host-memory tier [9, 35]	Idle conversations return cheaply	Transfer time counts in latency; host cap binds
Recurrent-state reuse and precision [36, 2]	Returning conversations skip recomputing the 48 recurrent layers; smaller per-conversation state	Drift from lower-precision state; stale or misaligned snapshots

Weight quantization and speculative decoding are out of scope because weights and model files are frozen. RamCove is not GPU rental, model training or a generic speed contest: a pure compute speedup is not the target, and the memory evidence of Section 8.3 is how the rules tell the two apart.

6.2 The first profile

A **profile** fixes everything that affects a raw capacity number; scores from different profiles never share a leaderboard. RamCove runs one competition per profile and opens profiles over time (Section 6.4); the first pairs Qwen3.8-27B with the H100 SXM.

Model: `Qwen/Qwen3.8-27B-FP8`, **revision** `017b9c7a…` (last modified 14 August 2026), Apache-2.0, 66 weight shards totalling 30,866,866,928 bytes whose SHA-256 hashes are frozen before submissions open [37]. Qwen describes Qwen3.8 as the most capable generation of its open models [5]. Its hybrid design gives the profile two memory resources instead of one: 16 full-attention layers with 4 KV heads of dimension 256 store exactly 65,536 bytes (64 KiB) of cache per token at 16-bit, and 48 Gated DeltaNet layers keep a fixed recurrent state of about 78.4 MB per conversation at 16-bit (153.9 MB with a 32-bit state) [1, 2]. The native context is 262,144 tokens, of which the profile serves 32,768; thinking is on by default and the profile turns it off per request [5]. The profile is text-only, so the built-in vision encoder does no work during measurement. About 29 GiB of weights leaves roughly 40 GiB for cache and recurrent state on one 80 GB GPU.

Engine: vLLM at an exact commit. The pin is the v0.29.0 tag, commit `98dff2a8…` (published 9 September 2026), with PyTorch 2.13.0 and the CUDA 13.0 build [38]. vLLM serves this model's architecture (`Qwen3_5ForConditionalGeneration`) from version 0.17.0, and v0.29.0 registers it [39, 40]. Prefix caching for hybrid models is still an open upstream item [36], which makes recurrent-state reuse one of the most valuable directions for miners. A version family is not a reproducible pin: patch releases, dependency resolution and container tags move. The profile therefore freezes the commit, the wheel or container image digest, every Python dependency by hash, the CUDA runtime, the GPU driver and firmware, the evaluator release and its container digest. The profile's calibration checks that the pinned stack serves the model correctly before the profile opens. Version 0.29.0 changed several defaults, so tuned settings are recorded explicitly rather than inherited [38].

Workload: a multi-turn document assistant (Section 8.5).

6.3 Hardware and execution

Authoritative evaluation worker. In the first profile, one complete **NVIDIA H100 SXM 80 GB** (the module form used on NVIDIA's HGX multi-GPU server boards: up to 700 W configurable, 3 TB/s memory bandwidth, NVLink) [41]. No MIG partitioning, time-slicing or any other workload on that GPU during measurement. The candidate VM receives 16 dedicated physical cores of a published CPU model on the GPU's NUMA node, 64 GiB of RAM, no swap and no network. These are **candidate limits**, not a server sizing: the physical host also needs memory and cores for the hypervisor, the trusted measurement agent and the load generator, and builds run on separate machines. The profile publishes the GPU memory ceiling (device memory − 1.5 GiB, sampled at the driver), the power limit (the SXM default, checked before every run), persistence and compute modes, driver and firmware versions, the CPU model and core allocation, clock source and synchronization, and every engine setting. Storage and log retention are planned at about 2 TB per worker-year and sized from calibration measurements.

Different hardware is a different profile. The H100 PCIe card has the same 80 GB but a 350 W limit, lower memory bandwidth and different interconnect; NVIDIA's own guide puts it at about 65% of the SXM's delivered performance [41, 42]. H100 SXM and PCIe results are therefore separate profiles, as are H200, B300 or RTX 5090 results. Each additional profile needs its own tuned baseline, noise study, calibration and published share of the miner allocation (Sections 6.4 and 9.3).

Evaluation GPUs versus inference GPUs. Adding workers lets the operator's evaluation finish more jobs at once. It never gives a candidate more hardware than its profile specifies: in the first profile every measurement is one model replica on one H100. Miners may develop on any equipment, but reliable local performance testing needs access to the profile's GPU (an H100 SXM for the first profile), and verified on-demand prices for an H100 SXM in containers or virtual machines ran from $3.49 to $6.88 per GPU-hour on 25 September 2026 [43, 44, 45]. Miners submit code; they never host customer inference.

Isolation boundary. A container on a shared host is not a defensible boundary for hostile native GPU code. NVIDIA Container Toolkit container escapes were published in 2024, 2025 and 2026, the latest fixed in version 1.19.1 [46, 47]; gVisor passes GPU driver calls through and is "much less effective" against driver vulnerabilities [48]; CUDA allocations are not cleared [49]. The boundary is:

1. **Builds** run in disposable, network-less VMs without GPUs.
2. **Measurement** runs each candidate in its own hardware-virtualized VM with the whole GPU passed through to the VM by the host's IOMMU and VFIO driver, so the VM talks to the device directly [50]. The authoritative measurement host runs nothing else untrusted at the same time. Other GPUs on the same HGX baseboard run candidates only if the profile's calibration shows that they do not disturb the GPU being measured; otherwise they stay idle or hold only trusted jobs, a cost the operating budget carries (Section 12.4).
3. **Between jobs** the VM is destroyed, the GPU is reset from the host and checked with a memory-pattern probe, and the host boots from a verified read-only image. A GPU whose probe finds data left from the previous job is taken out of service.
4. **Control** belongs to a separate trusted controller that holds the queue state and signing keys, receives measurements over a one-way channel, and never runs candidate code. Load generation runs outside the candidate VM.

Worker count adds throughput, not independence (Section 7.5). The boundary needs host-level control of the GPU: passthrough, reset and a verified boot image. A host serves as a measurement worker only when it provides that control and passes the reset and memory-probe checks. Public price lists do not state these conditions, so each host is qualified individually (Section 12.4). On 25 September 2026 the only listed bare-metal H100 SXM offer found was a whole 8-GPU node [51].

6.4 More models and GPUs

RamCove is not tied to one model or one GPU. Each profile is its own competition with its own tuned baseline, champion and share of the miner allocation, and new profiles open over time. A candidate profile needs four things: open weights under a license that allows a supported package; a pinned engine commit that serves the model correctly; a GPU type that the operator can dedicate whole to measurement; and teams that serve that combination. Opening one repeats the first profile's work: a tuning sweep for champion version 0, a noise study, calibration and a published allocation share, about 150 to 200 GPU-hours.

The next profiles follow customer demand and evaluation hardware. On the model side, the pinned engine already serves open families with different memory shapes, such as the dense Mistral Small 3.2 (Apache-2.0) and gpt-oss with its alternating banded and dense attention [52, 53, 54]; models under custom licenses get a license review first [55, 56]. On the GPU side, candidates include H200 and B300 data-centre GPUs and RTX

5090-class cards, each a separate profile because memory size and bandwidth change every number (Section 6.3). An improvement proven on one profile is ported and measured again on another, never assumed to carry over, and a method that wins on several profiles earns on each one it wins.

07 Network architecture, submission lifecycle and queue

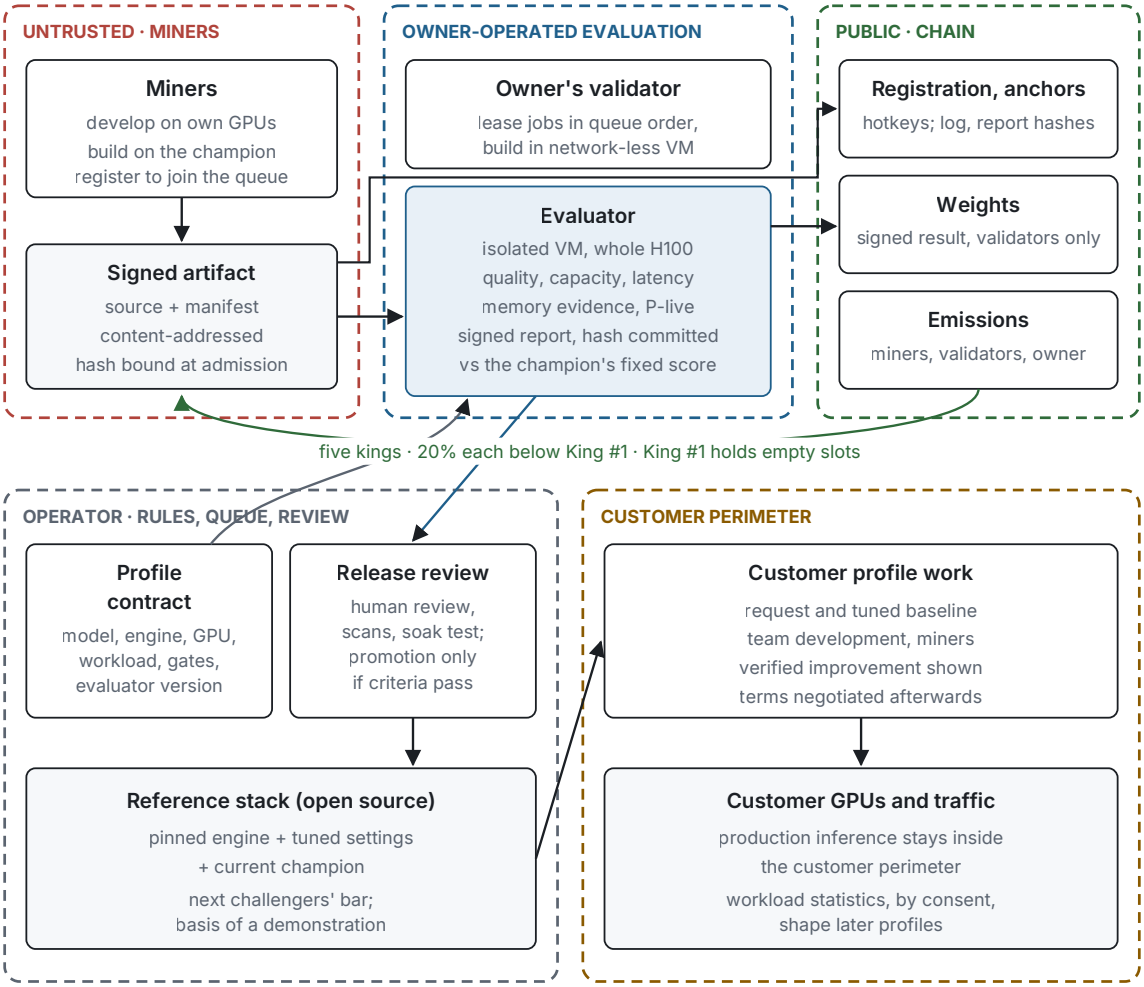


Figure 4. Architecture and trust boundaries. Miner code is untrusted and runs only inside isolated measurement VMs on the operator-run validator's workers. The intake service, queue and signing keys are trusted components kept off those hosts. The chain records registrations, anchors and weights, and the operator's signed weight result is served only to validators. Champions become the open reference stack. A customer's profile request leads to a tuned baseline, team-funded development and the miner competition, a verified improvement and a demonstration; commercial terms are negotiated only afterwards, and customer payments never touch the chain.

7.1 Roles and trust boundaries

Role	Provides	Receives	Pays for
Miner	Source patches; signed manifest	As King #2 to #5, 20% of the profile's miner allocation per slot; as King #1, the rest, including every empty slot's share (Section 9.1)	Own development hardware and time; subnet registration to join the queue
Operator-run validator	Runs every miner submission on the profile's specified hardware, checks correctness and performance, and signs the outcome as a weight result every tempo, served only to validators; on-chain weights	Validator share of emissions (shared with stakers)	Evaluation hardware, stake, operations
Other validators	Weights set from the operator's signed result, read with a request signed by their validator hotkey and checked for signature, chain and validity; no evaluation of their own	Validator share of emissions (shared with stakers)	A CPU machine and stake

Role	Provides	Receives	Pays for
Operator	Rules, evaluator releases, intake and queue service, tuned baselines and the team's reference improvements, review, demonstrations and customer delivery	Operator share of emissions (to the treasury); customer revenue under agreements	Engineering, review, its own GPUs, the evaluation of every miner submission, all work before a customer agrees
Customer	A profile request: requirements and workload statistics by consent	A demonstration on its configuration; under an agreement, delivery and further work it buys	No upfront fee; its own staff time and testing; after a demonstration, only what it agrees to
Staker	Stake that weights validators [57]	Staking returns; no revenue rights	Token purchase

An *evaluation worker* is one physical host with one whole H100 measured in isolation (Section 6.3). A *validator* is a registered key holder that sets weights. The operator-run validator also operates the workers: it runs miner submissions on the profile's specified hardware, checks correctness and performance, and signs the scores as a weight result that it serves only to validators. Other validators submit those scores as their weights. More workers finish more jobs; they are not independent checks of each other when one party runs them all.

7.2 The submission lifecycle

Every submission moves through the named states of Figure 5. A state change is written to an append-only, signed event log whose hash is anchored on-chain at least once per tempo (360 blocks, about 72 minutes) [58]. A revised artifact is always a new submission: it gets a new hash and a new place in line.

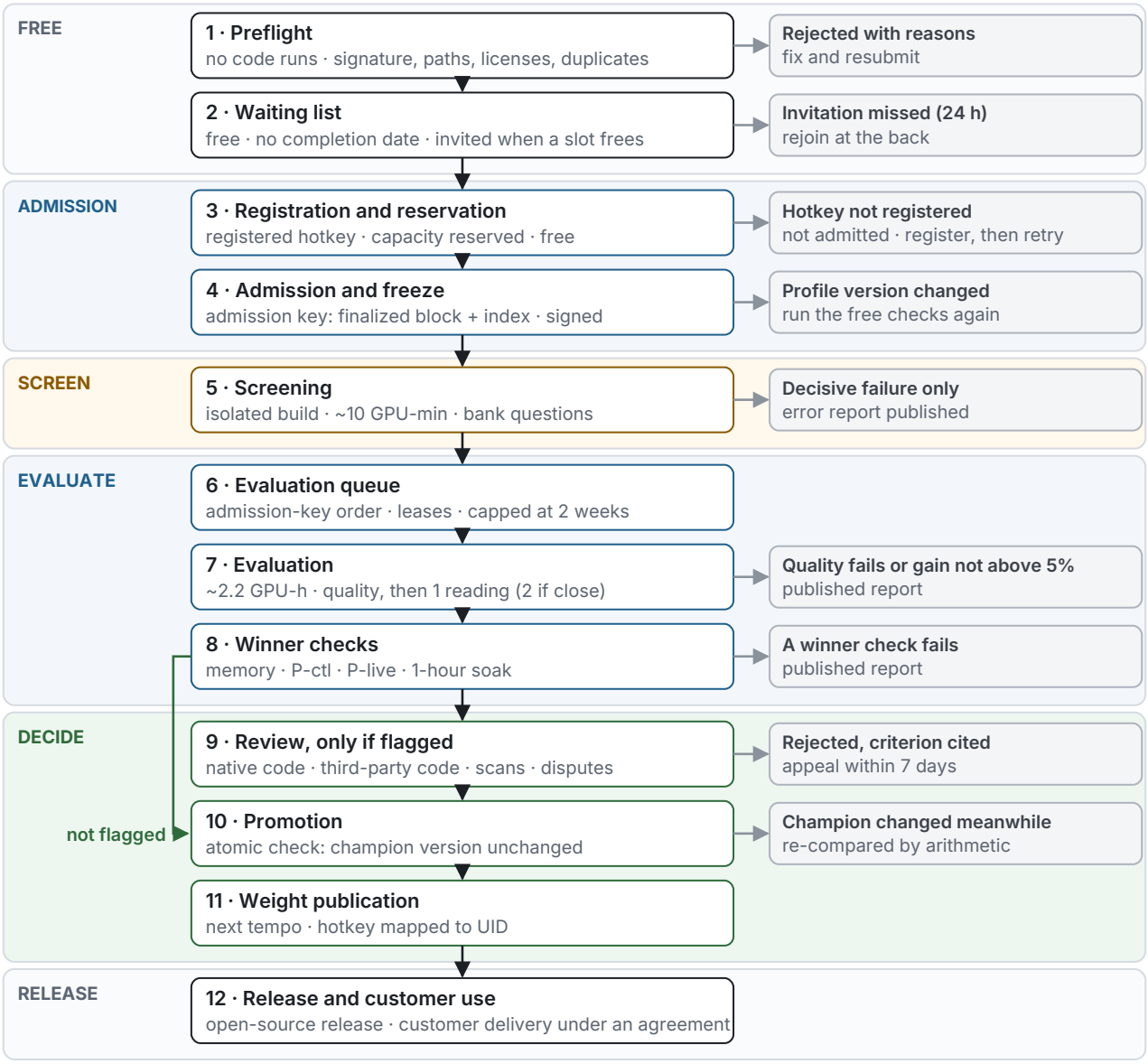


Figure 5. Submission lifecycle. Boxes are states; grey exits end a submission and name what follows. Submitting is free, and no untrusted code executes until the artifact and its admission are frozen. A short screen may reject only for decisive failures; it never crowns a winner. One evaluation decides each challenger. A winner is promoted in admission order after its winner checks. Its code is released as open source; it reaches a customer only through delivery under that customer's agreement.

1. **Preflight (free, rate-limited).** The public intake service checks, without executing anything: archive size (≤ 20 MiB by default), required manifest fields, the signature of the miner's *hotkey* (its Bittensor signing key), profile and evaluator version, that only allowed paths change, declared licenses, exact-duplicate hashes against every earlier submission and the champion, and that the hotkey has no other active submission and is not excluded. There is no daily limit; the only limit is one active submission per hotkey. Failures return machine-readable reasons.
2. **Waiting list (free).** If no evaluation slot can be reserved, a passing submission joins the profile's waiting list, ordered by the signed preflight time. A place on the list is free and carries no completion date. When a slot frees, the first entry receives an invitation valid for 24 hours; a missed invitation drops the entry, which may rejoin at the back.
3. **Registration and reservation.** Joining the queue needs the submitting hotkey to be registered on the subnet; the free checks do not. With a registered hotkey and a free slot, a reservation holds one slot and books the submission's expected work against the admission cap (Section 7.4). There is nothing to pay.
4. **Admission and freeze.** The intake service issues a **signed admission record** containing the artifact hash, profile version, hotkey and **admission key**: the number of the finalized block at admission and the submission's index among that block's admissions. The artifact is frozen from that moment and its place in line is fixed; the event log that records it is anchored on chain every tempo, so nobody can change the order afterwards.

5. **Screening (bounded).** A separate small queue builds the source in a network-less build VM (maximum 60 minutes) and runs a smoke screen of about 10 minutes of GPU time: engine start within 5 minutes, the 200 screening questions of the week's question bank (Section 8.6) sent in parallel, the output contract, and peak GPU memory at the ceiling. **Decisive failures** are: build failure reproduced on a second clean build; no start; crashes or more than 5% errors; output-contract failure on more than 5% of items; memory above the ceiling; or smoke accuracy more than 15 points below the champion's, a gap at least four standard errors wide on 200 items. Speed and capacity are not judged here. Each rejection publishes an error report with logs and hashes.
6. **Evaluation queue.** Screened submissions wait here in admission-key order (Section 7.4).
7. **Evaluation.** One worker checks quality on the week's bank questions, then takes one capacity reading of the challenger against the champion's fixed score on that worker (Section 8.8). A reading close to the margin gets one more. The planning estimate is about 2.2 GPU-hours.
8. **Winner checks, promotion, weights, release.** A winner (Section 9.2) passes the memory-evidence, short-request, multi-turn and soak checks (Sections 8.3, 8.5). It is then promoted in admission order (Section 10.5); where the profile version enables reproduction, a worker of a second evaluation group first reproduces its reading (Section 7.5). Promotion is an atomic check that the champion version compared against is still current. The operator signs the new weight result on the next tempo, and validators submit it. The champion's code is released once it has passed its gates and reproduction. Nothing deploys to customers automatically: a customer receives a build only through delivery under its agreement.

7.3 Admission and outcomes

Submitting is free from start to finish. RamCove requests no fee, issues no quote and takes no payment; the operator provides the GPU time of screening and evaluation. A submission needs only what the free checks ask, a hotkey registered on the subnet, and a free slot under the admission cap (Section 7.4). The admission key comes from the chain's finalized head when the intake service admits the submission, not from any transaction the miner sends, so nobody can buy or copy a place in line. Subnet registration is the chain's own, paid to the chain under its rules [59].

Event	Outcome (published rule)
Preflight rejection	Rejected with machine-readable reasons; fix and resubmit
Queue full	Free waiting list; invited when a slot frees; a missed invitation rejoins at the back
Hotkey not registered	Not admitted until the hotkey is registered; the miner can try again at once
Infrastructure fault, lease expiry or evaluator crash	Rerun at the same admission key; after a second fault the submission closes as an operator failure, which never counts against the miner
Miner-caused build failure or other decisive screen failure	Rejected, with a published error report
Evaluation completed, whatever the result	Result and report published
Champion changed while a winner waited (Section 7.4)	The winner is re-compared by arithmetic, not measured again
Flagged as a cosmetic copy	Rejected in the free checks; the rejection names the earlier submission it matches
Technically approved, but the hotkey was not registered within the grace period	Closed as expired; the incumbent keeps its allocation
Withdrawn by the miner while queued or under evaluation	Closed at once; its booked time returns to the queue

The admission cap, not a price, limits the GPU time the queue can claim, and registration is what a place in line costs a miner (Section 7.4).

7.4 Queue order, parallel workers and champion changes

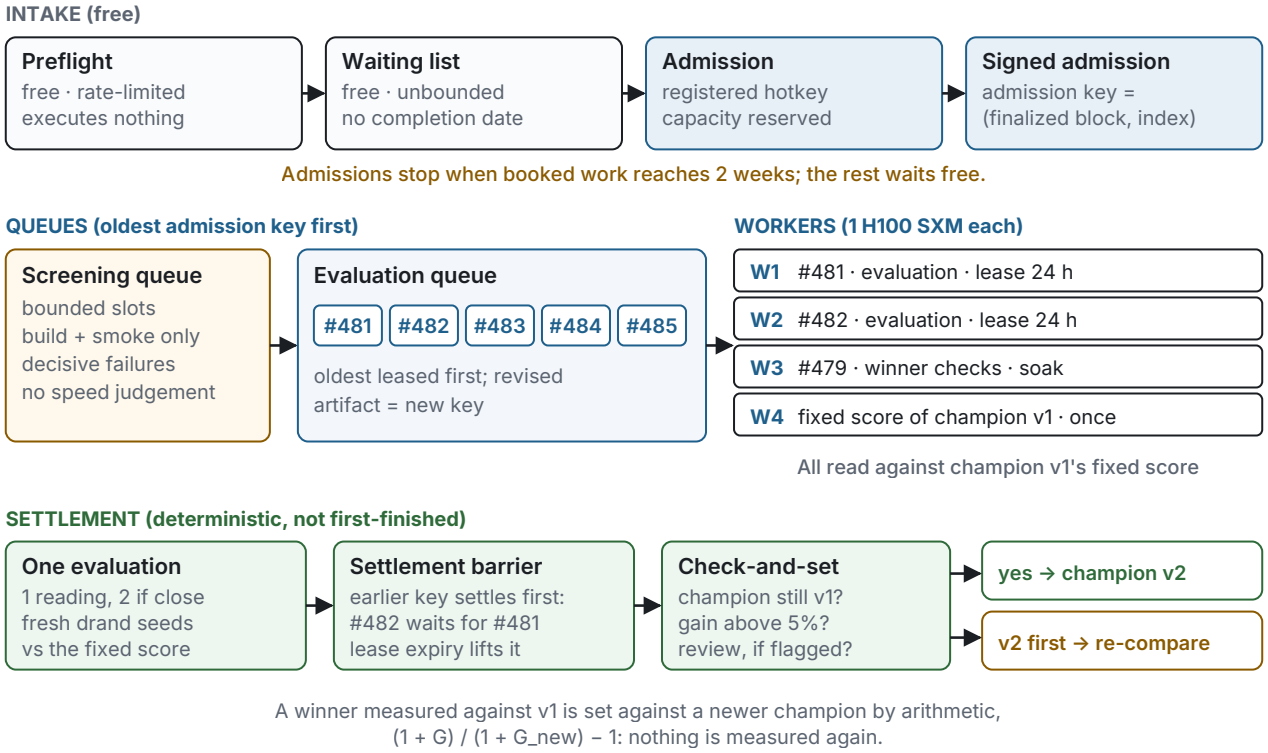


Figure 6. Queue and worker model. Admissions are capped by published capacity; further interest goes to the free waiting list. Workers lease jobs in admission-key order and may finish in any order. Every evaluation is read against the fixed score of the champion current when its lease began. Results settle in admission order, and promotion is an atomic check that this version is still current; a winner measured against an older champion is re-compared by arithmetic. Numbers are planning estimates.

Order. Each queue releases work oldest admission key first. The only published exceptions are objective: a lease that expired, an invalid submission, or a job needing a resource the free worker lacks, such as a different profile. Queue order is not completion order.

Limits. At most one active submission per hotkey and per lineage. *Active* means submitted and not yet final; a *lineage* is a declared chain of revisions, also inferred by similarity. Each admitted submission books the expected work of one evaluation (about 2.4 GPU-hours, Appendix B) until its result is known, and admissions stop when booked work reaches two weeks of published capacity. This limits how long the queue can grow; it is not a submission window, and submissions arrive at any time. Additional interest goes to the free waiting list. An admitted submission in the queue is always evaluated; waiting does not expire it. Anyone can create extra hotkeys, so the per-hotkey limit is not Sybil protection on its own; subnet registration, which costs every hotkey the chain's registration burn, the cap and similarity checks carry that weight. Similarity checks are fallible. Identical bytes are rejected in preflight, and so is a **cosmetic copy** (the same method after renaming, reformatting or reordering, caught by normalized diffs, syntax trees and behavioural fingerprints), which costs the miner nothing and names what it matched; and legitimate work that shares upstream open-source code or builds on the champion is allowed, because only a measured margin over the champion earns. A hash proves identity of bytes, never originality.

Leases. A worker takes a job on a lease: 2 hours for screening, 24 hours for an evaluation, 8 hours for a reproduction. A lease that expires is requeued once at the same key. A second expiry marks the job stuck and closes it as an operator failure, which never counts against the miner, so a hung job cannot block everyone else.

Settlement. An evaluation always compares against the champion version frozen when its lease began. Workers may evaluate several challengers against the same champion at once. Results settle in admission-key order, with a *settlement barrier*: a winner is not promoted while any earlier-admitted submission is still unfinished, and the barrier lifts when that earlier submission ends or its lease expires. The fastest-finishing job therefore gains nothing. Promotion is an atomic check-and-set on the champion version identifier.

When the champion changes while others wait. A challenger still in the queue simply runs against whichever champion is current when its turn comes; nothing extra is measured. A winner measured against an earlier champion can never dethrone the new one directly. It is **re-compared** by arithmetic: both gains are ratios to the same earlier fixed score, so its gain over the new champion is $(1 + G) / (1 + G_{new}) - 1$. If that still exceeds the

margin it keeps its place and admission key; otherwise it ends as not promoted. Nothing is measured again. Only if the published rules changed in between, or a rollback restored an earlier champion, is the evaluation run again, at the same key.

Visibility. A public status page shows each submission's queue position, state, champion version, admission key, failure reasons and a wait estimate labelled *estimate*. It shows no source code until promotion. A daily report shows backlog, worker availability and median waits. The daily operational target is to start screening within 24 hours for 90% of admissions; it promises nothing about completion times.

7.5 Who measures, and whose measurements count

Evaluation is operator-run. The operator-run validator runs every admitted submission on the profile's specified hardware, checks correctness and performance, and decides the king queue under the evaluation rules. Each of its workers measures a new champion once, when it is crowned, and every evaluation on that worker is set against that fixed score (Section 8.8). Workers run by one party are *parallel*, not *independent*: they catch hardware faults and noise, not that party's bias. Each report is signed and its hash committed before it is revealed.

Every validator sets weights from the operator's signed result. Every tempo the operator's controller signs a weight result built from the king queue: the share of the miner allocation each hotkey receives (20% for each king below King #1, the rest to King #1, and the whole allocation to the burn hotkey while no king exists), the chain and netuid, and a validity window of two tempos. **Only validators can read it.** Each read is signed with the reader's hotkey over the hotkey, method, path and a timestamp within 120 seconds of the operator's clock, and the operator serves the result only to a hotkey holding Bittensor's validator permit on the subnet. A validator, the operator's included, runs a CPU-only program that signs that read, checks the result's signature against the operator key pinned for the network, checks that the result is for this chain and subnet and still valid at the finalized head, maps the hotkeys to UIDs from its own chain read, and submits the weights. A missing, invalid or expired result means no submission; the program never invents weights. Validators other than the operator run no H100 tests: they rely on the operator's scores rather than checking performance themselves. The public site shows the king queue and the weight plan's status, not the weights by hotkey; the weights validators set are visible on chain, as on every subnet, once commit-reveal reveals them. A validator can instead delegate stake weight to the operator's validator with Bittensor's child hotkeys and run nothing at all.

Reproduction. A profile version can require a winner's reading to be repeated once by a worker of a second evaluation group, against that worker's fixed score, before promotion; that reading must also exceed the margin. It is off by default, because turning it on needs a second evaluation group. When it is on and the two readings disagree beyond the noise band, promotion holds and the reports go to the dispute panel (Section 9.6).

Checks anyone can make. Every replay input is published. Gate arithmetic, readings, the replacement rule, settlement order and the king queue can be recomputed exactly, and each week's question bank can be rebuilt from its revealed seed (Section 10.3). A validator can also check the signed weight result against that king queue.

Enforcement limits. Signatures and commitments prove who reported what and that it was not altered, not that the measurement was honest or free of collusion. On-chain, each validator sets its own weights, and the chain combines them by stake with clipping [57]. With operator-run evaluation the operator's scores decide the outcome (Section 10.5). Reproduction, where enabled, uses extra GPU-hours, which the cost categories of Section 12.3 include.

7.6 Timing: what takes how long

Interval	What sets it	Planning value
Arrival to preflight result	Intake service	Minutes
Waiting list	Demand versus published capacity	No promise; published estimate
Admission	Intake service; chain finality; the miner's registration	Minutes, once a slot is free and the hotkey is registered
Screening queue and run	Screening slots	Target start within 24 h; about 10 GPU-minutes plus a build
Evaluation queue	Backlog ÷ workers (Appendix B)	Hours to days; admissions pause at two weeks of booked work
Evaluation	GPU execution	About 2.2 GPU-hours (early failures about 1.2)

Interval	What sets it	Planning value
Champion's fixed score	Each worker, once, when a champion is crowned, with its answers to the current bank	About 6.5 GPU-hours per worker
Question bank	Operator platform, every 7 days (Section 8.6)	Fingerprint a day ahead; the champion answers each bank once per worker (0.75 GPU-hours)
Winner checks	Memory evidence, P-ctl, P-live and a 1-hour soak	About 3.6 GPU-hours, the same day
Reproduction, where the profile enables it	One reading by a second evaluation group	About 1.3 GPU-hours
Weight publication	The operator signs a new weight result on the next controller tick; validators read it, with a signed request, every 30 seconds	Within one to two tempos after promotion
Customer request to demonstration	Assessment, tuned baseline, team-funded development and the competition, a verified improvement (Section 4.1)	No promise: research time is unknown and a request may find nothing (Section 12.2)

7.7 On-chain and off-chain

On-chain: hotkey registration (needed to join the queue and to receive weight, Section 9.4) [59]; finalized blocks, which fix admission keys; commitments anchoring event-log and report hashes, rate-limited per account [60, 61]; weights, consensus and emissions [62]. **Off-chain:** artifacts (private until promotion), admission records, queues, evaluator releases, reports and logs, the signed weight result, reviews, disputes, and everything about customers. The chain registers hotkeys, finalizes blocks and pays emissions; it does not run evaluations or enforce any rule in this chapter beyond the registration and finality the intake service reads from it.

08 Evaluation: what is measured and how

A miner's own benchmark earns nothing. The operator-run validator builds the code, runs it against conversations generated after it was frozen, and measures how much more qualified work the fixed GPU serves than the current champion. Thresholds below are defaults; each profile's calibration sets the final values from measured noise and freezes them before submissions open (Appendix B). Queue simplifications never weaken these checks.

8.1 Quantities, kept separate

Six quantities are easily confused. *Model-weight memory* is frozen for the profile. *Cache memory* (the attention layers' KiB per token plus each conversation's recurrent state) is what a challenger shrinks or uses better. *Working memory* (activations, buffers, allocator slack) may shrink too, but the whole-process total is capped. *Resident context capacity* is how many tokens fit at once: memory fit, necessary but not sufficient. *Throughput* counts completed tokens or requests regardless of limits and can rise while service worsens. **Qualified serving capacity** is the comparison quantity. Halving bytes per token does not halve any cost by itself: compute or latency limits may bind first, and a smaller cache may lose needed information.

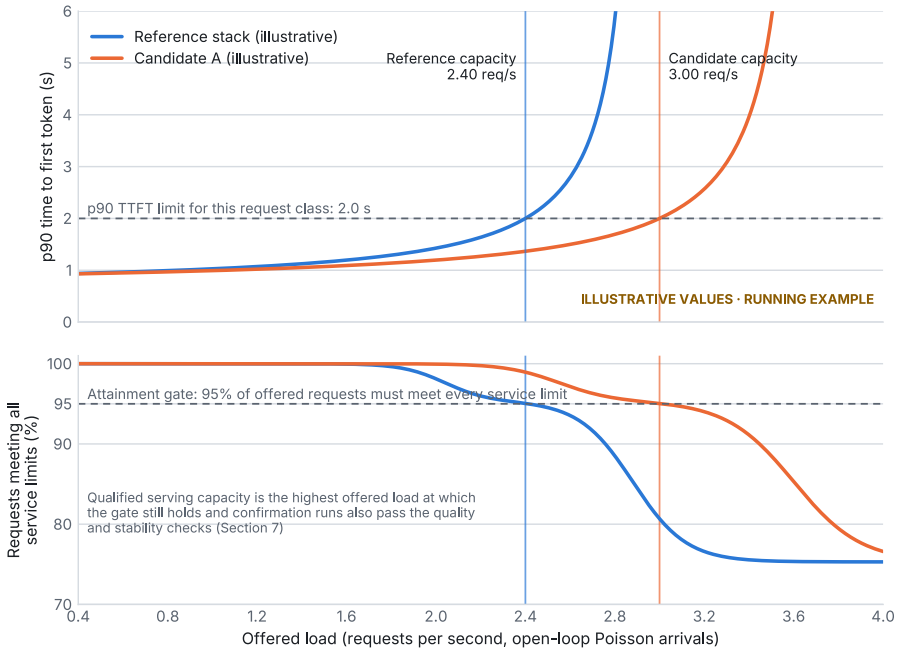


Figure 7. Capacity is set by service limits, not raw throughput. Illustrative curves for the example champion and challenger. Top: p90 time to first token (the value 90% of requests beat) rises with offered load and crosses this request class's limit. Bottom: the share of requests meeting every limit falls through the 95% gate at the same load. Capacity is the highest load left of that crossing whose runs also pass the quality and stability gates. Values are illustrative.

8.2 Qualified serving capacity

> **DEFINITION**
>
> **Qualified serving capacity** is the highest offered request rate (open-loop Poisson arrivals, requests per second) at which a stack passes every gate. The champion's capacity, its **fixed score**, is *confirmed* once per worker when it is crowned: three fresh 20-minute runs each pass the per-run gates and together pass the pooled gates. A challenger's **reading** is the highest rate on a published grid, stepping up from that fixed score, at which one run passes the per-run gates (Section 8.8).
> **Qualified request:** no error; first non-whitespace token within its time-to-first-token (TTFT) limit; mean time per output token (TPOT) within limit and no gap above the stall limit; finished within a length-aware end-to-end limit and a 60 s deadline; output meets its task contract.
> **Per-run gates:** ≥ 95% of offered requests qualified; ≤ 0.5% errored, rejected or past deadline; no upward latency drift; recovery after bursts; in-flight requests drain; memory, host RAM and network ceilings respected.
> **Pooled gates:** ≥ 90% qualified in every context band and request class across the passing runs; accuracy under load non-inferior to the champion's; output lengths not shorter than the champion's. For a winner, accuracy and length are pooled over its 1-hour soak.

Every scheduled request counts, including rejected, dropped or unfinished ones. Load is offered on a fixed timeline regardless of server state, because closed-loop generators hide overload [23]; this follows the reference benchmark's server scenario [21] and serving research's goodput definition [22]. Per-band attainment stops a scheduler from pushing the longest requests to the back.

8.3 Memory evidence, and what it cannot prove

RamCove's target is capacity gained by using memory better. No single test proves that a gain was *caused* by memory, so every profile requires three kinds of evidence and names what each cannot do.

1. **Tighter-budget retention.** The ceiling drops by a quarter of the cache budget (40 to 30 GiB in the example). The champion's fixed score includes its capacity at that budget, measured once when it was crowned, and the challenger's reading at the tighter budget must beat it by more than the replacement margin. A tighter reading that misses the margin gets one more with fresh traffic, and either may pass, so one noisy run cannot fail a winner here. A memory method usually keeps or grows its advantage as memory gets scarcer, while a compute speedup tends to shrink. *Limit:* a mixed change, or a speedup that also lowers working memory, can pass.
2. **External memory accounting.** Measured from outside the engine: the largest set of idle conversations that resume without recomputation (detected by their first-token time) at the ceiling, and prefill tokens recomputed per qualified request under load. Engine counters corroborate but never decide. The resident-

conversation figure must improve by the replacement margin; recomputed prefill per qualified request is published beside it but does not by itself qualify a winner, because it can also fall for scheduling reasons. That is the probe, and not the tighter-budget reading, that rejects a compute-only speedup.

3. **Mechanism declaration.** The manifest declares whether the change acts on cache storage, retention, allocation or admission. The declaration is published with every winner and open to dispute (Section 9.6), and a reviewer confirms it in the code whenever the winner is flagged or disputed (Section 10.5). *Limit:* human judgement, and only when flagged or disputed.

Validation before use. In each profile's calibration the diagnostics run against at least three known memory methods (built-in 8-bit cache against 16-bit, a prefix-retention policy, capped host offload) and three known compute-only changes (an alternative attention backend, graph-capture settings, scheduler token budget). They are adopted only if every memory method passes and no compute-only change does. Otherwise the profile's objective is stated as *capacity under memory pressure*, and speedups that win under that workload are accepted as such. A short-request control guards against regressions: P-ctl is one run of short requests at the rate recorded with the champion's fixed score, and its attainment may fall at most 2 points below the champion's. Adaptivity driven by live memory occupancy, queue length and request state is permitted; branching on features that identify the evaluation is forbidden, checked in review when a winner is flagged, and grounds for a dispute at any time.

8.4 The profile contract

A versioned, hashed published specification, not a smart contract (full JSON in Appendix A).

Identical for champion and every challenger	Challenger may change, within allowed paths
Model revision, weight hashes, tokenizer, template, non-thinking mode; engine commit, image digests, driver, firmware	Cache layout, data types, scaling; eviction, retention, admission; batching; cache and attention kernels; allocator
One GPU of the profile's type (H100 SXM in the first profile); GPU ceiling = device memory − 1.5 GiB, sampled at the driver; candidate VM with 16 cores, 64 GiB, no swap, no network	Listed engine settings, including memory utilization, while the ceiling holds
Context limit 32,768; greedy decoding; workloads, limits, windows, bursts; evaluator, grader, thresholds, margin, admission rules	Nothing else

The ceiling is enforced from outside with per-process driver accounting [63], because engine counters can be modified, and the tuned baseline uses the same ceiling so no headroom is free. Host RAM is capped so spilling cache into system memory is not a GPU saving; transfers count in timing.

8.5 A realistic workload

P-mem is a multi-turn enterprise document assistant generated from public templates, the week's question-bank seed and a per-reading seed (Appendix B). Conversations have 1 to 6 turns (about 60% of requests are follow-ups). The first turn carries a shared 400-token system prompt and up to three synthetic documents with planted facts. Context at request time is 25% under 2,000 tokens, 45% at 2,000–8,000, 25% at 8,000–24,000 and 5% at 24,000–32,000. Graded questions target facts early, middle and late in the context, including relational and negation questions, with unasked distractors. Users think for a median 30 seconds between turns, so memory binds: at 2.40 req/s (illustrative) about 72 open conversations want about 41 GiB resident (about 580,000 tokens of attention cache plus 72 recurrent states) against a 40 GiB budget. Arrivals are open-loop Poisson with two one-minute bursts at 1.5× [64]. Warmup uses separate-seed conversations, so no test prompt is replayed.

Known limitation of prepared histories. In P-mem, earlier assistant turns are prepared text containing planted facts, not the model's actual earlier answers. This keeps load open-loop and grading exact, but it misses errors that compound across turns and the real length of earlier answers.

P-live, a separate check with actual responses. New conversations still arrive on a fixed Poisson schedule. Each later turn is built from the model's *actual* previous answer and sent after that answer completes plus a scripted think time. Slow systems cannot hide demand: every scheduled conversation counts from its arrival, and when an answer misses its deadline, every remaining turn of that conversation counts as offered and unqualified. Graded questions ask about documents and about the model's own earlier answers, checked against its logged text. P-live runs once for 20 minutes at 90% of the challenger's measured P-mem capacity and must meet the per-run gates. It does not define capacity, because its offered load depends on the system under test.

8.6 Quality, made testable

All grading is deterministic (exact, numeric or set match); plausibility, length or likelihood is not evidence, and judge models only flag cases for humans.

The operator's question bank. Validators do not write the questions. The operator platform builds one secret bank per profile every 7 days: the 200 screening questions, the 4,000 offline questions and the traffic scripts that capacity readings play. Each week's seed comes from a secret only the operator platform holds. A day before the week starts, the bank's fingerprint (a hash of its contents) is published in the event log that is anchored on-chain. During the week, the evaluation workers receive that week's seed with each job, never the secret itself, so every submission in the week answers the same questions. When the week ends, the seed is published and anyone can rebuild the bank and check it against the fingerprint. *Limit:* during a week the evaluation workers hold its questions, so whoever operates them could leak them to a miner; the weekly rotation, the public reveal and the per-reading beacon round limit what a leak is worth, but do not prevent it.

1. **Offline paired suite:** 4,000 items from the week's question bank (3,200 long-context retrieval items in the style of the RULER benchmark [65], stratified to 800 per context band and about 1,067 per position; 800 general), run at concurrency 32 under the tighter budget so compression paths are exercised.
2. **Known answers under load:** about 930 graded items per 20-minute run. The champion's three confirmation runs (about 2,780 items) are recorded with its fixed score, and a winner's 1-hour soak at 90% load supplies about 2,500; unanswered items count as wrong.
3. **Output contract** on every response, plus a length gate: answers at least 90% as long as the frozen reference's, so a stack cannot pass by answering more briefly.

Non-inferiority uses a margin fixed in advance and a one-sided 97.5% bound [66, 67] on paired question-level differences with a paired bootstrap [68, 69]. Offline, the bound must exceed −2.0 points overall and −3.0 in each position and band. Under load, the point estimate must exceed −2.0 and the bound −3.0. The margins are frozen with the profile before submissions open. A customer's own quality requirements are checked separately, in its demonstration (Section 4.3).

Two floors under the paired comparison. A challenger only has to be no worse than the champion, and a chain of champions could each lose a permitted fraction of a point until the answers are poor. So the same answers are also compared with a **frozen quality reference** — the tuned baseline at 16-bit cache, fixed for the profile version — with a bound of −3.0 points, and they must clear **absolute floors**: 85% overall and on general questions, 80% in each position group. The reference never changes within a profile version, so these floors do not drift with the crown.

8.7 Resources and failures

The evaluator samples per-process GPU memory at the driver every 100 ms and compares the peak with the ceiling; allocator counters are informational, because a caching allocator can hold freed memory that still looks used [70]. All timing is taken at the load generator, so queueing, compression and transfers count. Initial limits: TTFT 1.0 s + 0.1 s per 1,000 prompt tokens; mean TPOT 100 ms; stall 2.0 s; end-to-end TTFT limit + 0.15 s per output token; deadline 60 s. They are reset so the tuned baseline passes at 80% of its capacity. Runs are 8 minutes of warmup, a 20-minute window (8 minutes for each rate of a reading) and up to 90 seconds of drain. p90 end-to-end time in the last quarter may not exceed 1.40× the second quarter's, the over-saturation signal load tools use [71], and must return to 1.35× the pre-burst level in the quiet stretch after each burst. Both bands are set by the window's sample size, not by taste: an 8-minute window leaves about 60 requests per quarter and about 30 on each side of a burst, and tighter bands, such as 1.25× and 1.20×, fire on that sample's own noise in one run in five well below capacity. The wider bands fire in about one run in twenty and still reject over-saturated runs, where attainment fails as well. Both checks are ratios, so both apply only while the later p90 end-to-end is at least half of the requests' own limits: a stack with memory to spare answers so quickly before a burst that a harmless rise afterwards becomes a large ratio, and without that floor a stack 30% better than the champion failed as often as the champion at every load.

8.8 One evaluation against the champion's fixed score

The champion's fixed score. When a champion is crowned, each worker measures it once and records the result. Capacity is searched from its predecessor's score, rising in coarse jumps until a failure and bisecting to one grid step (3%); because a search picks the best of noisy outcomes [72], three fresh 20-minute runs then confirm it one step below the edge, and if any of them fails the confirmation moves down another step and tries again, as far as the grid allows. A score that still does not confirm is searched again from a quarter lower, twice at

most: a worker without a fixed score can judge nobody. The same is done at the tighter budget, and the short-request baseline is recorded. This takes about 5.8 GPU-hours per worker. The score is not measured again for later challengers: every worker runs the same frozen profile on the same GPU type, so a challenger measured later on that worker faces the same conditions. It is recorded per worker because physical GPUs differ slightly. A driver, firmware, image or evaluator change is a new profile version, and the champion is measured again then.

One evaluation. Offline quality comes first; failing it ends the evaluation. The challenger then takes one **reading**: an 8-minute run at the champion's fixed score, then one grid step (3%) higher after every run that passes the per-run gates. The reading is the highest grid rate that passed. It climbs until the first rate that fails, up to the top of the published grid, 60% above the fixed score; it is not cut short, because a winner's gain may later be set against a newer champion by arithmetic (Section 7.4). A reading at or below the margin loses, and one at or above the clear-win line of 9% wins. A reading in between gets one more reading with fresh tests, and the average of the two decides (Section 9.2). A winner then passes the **winner checks**: pooled band attainment across its passing runs, memory evidence (Section 8.3), P-ctl, P-live (Section 8.5) and a 1-hour soak at 90% of its measured capacity with no errors above the gate. An evaluation takes about 2.2 GPU-hours, and a winner's checks about 3.6 more.

Per-comparison seeds. The questions come from the week's bank; the exact traffic of each reading also depends on a beacon round. Before a reading starts, the controller writes a comparison record into the event log: champion version, challenger hash, worker, and a future drand round at least 30 minutes after the record's on-chain anchor is finalized. The reading's seed is the hash of that round's randomness and the comparison identifier, mixed with the bank's traffic scripts. drand is unpredictable and unbiasable below its collusion threshold [73], so neither party can know or redraw the tests. A missing round rolls to the next round by a fixed rule. A reading that fails for infrastructure reasons is voided and published, never silently discarded; its tests have been exposed, so a replacement reading gets a new record and a new future round. A reading delayed past its lease is rerun with fresh tests rather than reusing exposed ones.

What is reused. Every evaluation on a worker reuses that worker's fixed score of the champion until the next champion is crowned or the profile version changes. Because the questions are fixed for a week, the champion answers each week's bank once per worker, with its fixed score or at the first evaluation after the bank rotates; every evaluation that week compares the challenger's answers with those, item by item. A challenger's readings are never reused for another submission.

8.9 Strong baselines and the moving champion

Before a profile opens, the operator publishes a tuning sweep of built-in features (memory utilization to the ceiling; 8-bit cache with default and calibrated scales; prefix caching; chunked prefill; block size; batch limits; capped offloading) [8, 9, 30] and adopts the best qualifying setting as **champion version 0**, which has no author and earns nothing. If built-in 8-bit storage passes the quality gates, challengers must beat 32 KiB per token, not 64. Qualifying open-source cache policies are folded into version 0 first. Version 0 is the tuned baseline the profile launches with; no improvement has to exist before a profile opens. Improvements the team develops later are reference work, attributed to the team, and enter only as an announced new profile version (Section 4.3). Afterwards the champion is always the stack every challenger is measured against, so improvements accumulate.

8.10 Illustrative worked example

ILLUSTRATIVE WORKED EXAMPLE (CONTINUES SECTION 4.6)

Champion v0 (tuned stack, built-in 8-bit cache failed the early-position stratum): fixed score **2.40 req/s**. Six challengers are admitted in the order E, A, D, F, C, B.
E · fragmentation-aware allocation. First reading +2 grid steps (+6.1%): between 5% and 9%, so one more reading, +3 steps (+9.3%). The average, **+7.7%**, exceeds 5%; the winner checks pass and nothing is flagged. **E becomes champion v1** and King #1, with a fixed score of 2.60 req/s measured on crowning.
A · 8-bit entries with online scaling and conversation-aware retention. Evaluated against v0 while E's checks were still running: reading +7 steps (+23.0%), a clear win. Its native kernel is flagged for review. E was admitted first and is promoted first, so A is re-compared by arithmetic: 1.230 / 1.077 − 1 = **+14.2%** over v1, still above 5%. Review passes. **A becomes champion v2** and King #1, with a fixed score of 3.00 req/s; E moves down to King #2 and keeps earning its 20%.
D · block-layout change. Reading +1 step (+3.0%): at or below 5%, so it loses on one reading.
F · faster decode-attention kernel. Reading +9.3%, but its tighter-budget reading is only +1% over the champion's and external accounting shows no change: memory evidence fails.
C · 2-bit entries with per-access decompression. Offline pass, but only 88% of requests qualify at the champion's fixed score: the first rate fails, so it is below the champion.
B · eviction to a quarter of the cache. Offline −8.5 points: fails after about 1.5 GPU-hours, a real memory saving customers cannot use.

09 Champion rule and incentives

9.1 King of the hill within a profile

Each profile keeps a queue of up to five **kings**. **King #1** is the approved **champion**, the stack every challenger must beat. A new champion is crowned only by the replacement rule of Section 9.2 plus every gate of Section 8. The challenger that beats King #1 becomes the new King #1, every king moves down one place, and King #5 leaves the queue and is paid no longer. Each king below King #1 earns **20%** of the profile's allocated miner weight while it stays eligible, and **King #1 earns the rest**: its own 20% plus the share of every empty slot. As the queue fills, King #1's share falls from 100% (alone) to 80%, 60%, 40% and finally 20%, when all five slots are held. Before any challenger has beaten the tuned baseline there is no king, and the whole allocation burns (Section 9.3). The rule pays nobody outside the queue: no runner-up pool, no maintenance share, no split for near-ties. Ties and unclear results keep the incumbent.

This choice is deliberate, with known costs. **Unpaid work:** a challenger that improves the champion by 4% earns nothing, however real the gain. **Long-lived kings:** a strong champion may hold the top place for months, with no calendar reset, and the kings below it keep their places as long as no new king is crowned. **Small complementary gains** go unrewarded unless someone combines enough of them to clear the margin. **Incumbent advantage:** the champion's author knows the code best. What limits these costs: the champion's code is public after promotion, so anyone can build on it; the margin is tied to measurement noise rather than set high for protection; and each new king takes the top place, and with it the largest share, as soon as it clears the bar, while the kings it displaces keep earning 20% each until they fall out of the queue.

9.2 The replacement rule

```
G   = R_challenger / S_champion - 1     one reading R against the champion's fixed score S, same worker
G ≤ δ          the champion stays
G ≥ δ_clear    the challenger wins
otherwise      one more reading G₂ with fresh tests, on its own beacon round two hours after the
first's;
               the challenger wins only if (G + G₂) / 2 > δ
A winner must also pass every quality gate and winner check (Section 8)
Default values:  δ = 5%,  δ_clear = 9%,  grid step 3%,  grid up to 60% above S
```

This is the only replacement margin in the paper. S is the champion's fixed score on the worker that takes the reading, confirmed with three fresh runs when the champion was crowned (Section 8.8). R is the challenger's reading on the same worker: the same hardware, limits and quality requirements, on tests generated fresh for that reading. Gains are compared in whole parts per million, so a gain of exactly 5% does not win. On the 3% grid the first rate above the margin is +6.1% and the first at the clear-win line is +9.3%, so one reading decides

whenever the result is clearly above or below the margin, and a second reading is spent only near it. In a simple model with 2% run-to-run noise (each profile's calibration substitutes its measured noise), a true 3% gain wins about 0.1% of the time, a true 5% gain about 7%, a true 8% gain about 69% and a true 10% gain about 95%.

Repeated attempts. Each attempt needs a new submission and fresh tests. Readings are never pooled across attempts, and every attempt's result is published. Because a win needs a reading above the margin, re-rolling a true 3% change almost never succeeds.

Freezing the rule. The profile's calibration sets δ, the clear-win line and the grid step from the measured noise so that a true 10% gain wins at least 80% of the time and a true 3% gain almost never does. The values are published with the profile and frozen before submissions open. A change applies only to submissions admitted after its announcement, never to one already in the queue.

> **ILLUSTRATIVE CHAMPION-VERSUS-CHALLENGER EXAMPLE**
>
> The champion's fixed score is **100 requests per minute**; the grid reads 100, 103, 106.1, 109.3, 112.6, 115.9.
> **Challenger at 103.** Its reading is 103 (+3.0%), at or below 5%. **Fails on one reading.** The incumbent stays.
> **Challenger at 107.** First reading 106.1 (+6.1%), close to the margin, so one more reading. If it reads 106.1 again, the average of +6.1% exceeds 5% and it wins; if it reads 103, the average of +4.5% does not.
> **Challenger at 110.** Reading 109.3 (+9.3%): **a clear win on one reading.** It is promoted only if quality, response-time limits, resource ceilings, memory evidence and the other winner checks also pass.
> **Challenger at 140 with unacceptable quality.** Offline accuracy falls below the non-inferiority bound. **Fails**; capacity is never considered.
> **Identical copy of the champion.** Rejected in free preflight by hash. A cosmetic copy is flagged by similarity; if it slipped through, its reading would be about 100 (+0%). **No new crown.**

9.3 From champion to weights

While the first profile is the only one open, it receives 100% of the weight validators assign to miners. As profiles are added, the split across them (for example 60/40) is published before any submission to a new profile is accepted, and the king queue applies inside each share. Weights steer the miners' part of emissions only, currently 41% of the subnet's alpha emission [57]. Kings never receive operator or validator emissions, and a weight never guarantees an exact token amount: the realized amount depends on the subnet's emission, stake-weighted consensus across validators and the alpha price.

Burned shares. When a profile opens there is no king, and the whole allocation goes to the subnet owner's UID. Once a king exists, the shares of the empty slots go to King #1, so nothing burns while every king is eligible. The share of a king who is ineligible or not registered goes to the owner's UID; when that king is King #1, the empty slots' shares go with it. Under the chain code running on the access date, miner emission sent to owner-associated hotkeys is burned by default, and is recycled instead only if the owner changes a per-subnet setting [74, 75]. The operator does not self-mine, no starting winner is invented, and operator-affiliated hotkeys cannot be champions. The burn is visible on-chain, but it is not free: the chain multiplies the subnet's share of TAO emission by one minus the burned fraction [75]. A long period without a king is therefore costly. The operator re-checks this behaviour whenever the chain runtime is upgraded, because these rules changed twice in August 2026.

Weight rhythm. The operator signs a weight result every tempo, and every validator submits it (Section 7.5). Hotkeys are mapped to UIDs (numbered subnet slots) again at every weight-setting, each tempo, so a reward follows the hotkey, never an old UID number that another miner could later occupy.

9.4 Registration, attribution and retirement

Registration. The free checks need only a hotkey signature. Joining the queue needs the hotkey to be registered on the subnet: with no submission fee, registration is what a place in line costs. A registered hotkey can still be pruned during a long wait, because registration immunity cannot cover weeks in a queue: the default is 4,096 blocks (about 14 hours), and the largest value the operator can set, 65,535 blocks, is about 9 days, so immunity cannot cover a 14-day grace either; after it, the lowest-emission non-immune UID is pruned, and a new hotkey can take the same UID number [76, 77, 78]. A pending submission keeps its identity (hotkey, artifact hash, admission key) whatever happens to UIDs. To receive weight a king's hotkey must be registered on the subnet, so a winner whose hotkey was pruned while it waited is **not promoted until it registers again**: a cleared submission waits in admission order, keeps its place, and has 14 days to register the **same** hotkey. Its evaluation is never repeated. If the grace ends first, the submission closes as *expired: hotkey not registered in time*, and the incumbent

keeps the crown. A king that deregisters later has the same 14 days to come back; meanwhile it keeps its place, and its share is burned and never back-paid. Promoting first would put the crown on a hotkey that cannot be paid, and take it from an incumbent that can.

Loss of eligibility. A king whose hotkey is still unregistered when the 14-day grace ends, or that is excluded after a dispute or found unsafe, is **retired** and never regains payment. It keeps its place in the queue, and its share is burned until it falls out. A retired champion's code remains in the reference stack as the bar, so a copy cannot win by default.

Unsafe champions and rollback. If a security or correctness flaw appears after promotion, the champion is suspended immediately (its share burned) and the reference stack rolls back to the previous champion version. The rolled-back king leaves the queue and the kings below it move up one place, so if the flaw was not deliberate and the previous champion's author is still registered and eligible, that author is King #1 again. A deliberate flaw leads to exclusion of the hotkey and lineage.

Upstream improvements. When the pinned engine is upgraded, the champion's code is ported and recalibrated as a new profile version, announced in advance. If upstream has absorbed the improvement so the champion no longer beats the new baseline by the margin, the champion retires with notice and its share is burned until a challenger qualifies.

9.5 Reference stack, champion and customer deployments

The **reference stack** is the tuned engine plus the current champion's code. Because every challenger builds on the champion, the stack accumulates all accepted improvements with attribution in the manifest lineage. A promoted submission *becomes* the reference; merging does not erase its standing reward, which lasts until a successor wins. A successor that includes earlier champions' code earns only by clearing the margin over them, so an improvement is never rewarded twice. Licenses must be compatible with Apache-2.0. A **customer deployment** is a build of the reference stack prepared for one customer's configuration and delivered under its agreement (Section 4.1). A competition win is not the same as customer value (Section 4.3): a new champion reaches a customer only through delivery under that agreement, never automatically. Upstream security fixes are back-ported within 14 days.

9.6 Spam, copies and disputes

Admission control is Section 7.4: subnet registration, the capacity cap, one active submission per hotkey and lineage, similarity checks. Building on the champion, porting published methods with attribution, and using declared open-source libraries are permitted. Cosmetic copies, one method spread across many hotkeys, and repackaging upstream features are farming and grounds for exclusion. Any participant may dispute a promotion within 7 days of its publication; the panel (the operator plus one independent reviewer) rules within 10 business days. Remedies are prospective: suspension, exclusion, rollback. The chain has no miner slashing, and optional collateral is enforced only by scoring zero [79, 80].

10 Security, auditability and trust assumptions

10.1 Who is trusted for what

Party	Trusted for	Not trusted for
Miner	Nothing; code is hostile until measured in isolation	Timing, memory, quality or originality claims
Operator-run validator	Running miner submissions on the profile's specified hardware in isolation, checking correctness and performance, and publishing signed scores under the evaluation rules	Honest measurement by assertion: a signature proves who signed and that nothing changed, not that the numbers are true or uncolluded
Other validators	Submitting the operator's signed result after checking its signature, chain and validity	Independent verification: they do not re-run evaluations
Operator	Intake, queue, review, releases, burning unpaid allocation	Fairness by assertion; its hotkeys cannot be champions, and its rule changes apply only to later admissions
Chain	Registering hotkeys; finalizing the blocks that fix admission keys, and anchors; weights; emissions	Randomness, secrecy, correctness of reports, or enforcing any queue rule
drand beacon	Unbiasable randomness at a pre-committed round, unpredictable unless a threshold of nodes colludes [73, 81]	Secrecy after release

10.2 Isolation

Section 6.3 defines the boundary: network-less build VMs; one hardware-virtualized measurement VM per candidate with whole-GPU passthrough; nothing else untrusted on the measurement host; destroy, reset and verify between jobs; and a separate trusted controller holding keys and queue state. Artifacts, evaluator releases, admission records, weight results and reports are signed; keys live in hardware-backed stores on the controller and rotate on schedule and after any suspected compromise. A container alone is not treated as a security boundary. Periodic escape tests, and packet capture on the no-network rule, check the setup.

10.3 Randomness, secrecy and audit

Artifacts are frozen by their hash and signed admission record before any comparison record exists, and the champion version is frozen when a lease starts. Questions come from the operator's weekly bank, whose fingerprint is anchored before the week and whose seed is published after it (Section 8.6); the traffic of each reading also depends on a drand round fixed in an anchored record before that round exists (Section 8.8), so nobody can choose or redraw it. Block hashes would be weaker randomness because producers can influence them [82, 83]. Once a round is public anyone can regenerate the tests, which is harmless: the artifacts are frozen and runs are offline. Per-validator canary items from pre-committed salts detect behaviour that depends on content the artifact could not have seen. Templates are public, so task *kinds* are known, but the questions themselves rotate every week and stay secret until the week ends. Replaying published arithmetic (grading, gates, readings, the replacement rule, settlement order, weights) is exact; rerunning GPU measurements is noisy, and only disagreement beyond the published noise band is evidence.

10.4 Threat table

Threat	Defense	Residual risk	Test
Benchmark recognition	Tests generated after freeze; forbidden-adaptivity rule; review of flagged winners; disputes	Tuning to template structure	Reruns with an unpublished template family
Test leakage or chosen seeds	Operator's weekly bank, fingerprint anchored a day ahead, seed published after the week; future drand round per reading; no redraws; voided readings published	The evaluation operator leaking the week's bank; beacon threshold compromise	Rebuild each revealed bank against its fingerprint; audit record-to-round timing
Cherry-picking runs	All requests count; fresh tests for every reading; the champion's fixed score confirmed with three fresh runs and published before any challenger is read against it	Validator bias (Section 10.5)	Replay gates and readings from logs
Dropping, delaying, shortening	Deadlines; per-band attainment; stall, drift, drain and length gates; P-live counts every remaining turn	Mild favouring of easy requests	Inject known-difficulty requests
Fake memory or timing; hidden offload	Driver-level sampling; client-side timing [63]; VM RAM cap, no swap, no network	Peaks under 100 ms; misconfiguration	Probe allocation must be seen; test candidate must fail
Queue-jumping or a forged admission	Admission key from the finalized block and an index within it, set by the intake service; signed admission records in the anchored event log	Operator database error	Replay admission order from the event log and its on-chain anchors
Queue jumping, fastest-finish advantage, stuck jobs	Admission key from finalized chain order; settlement barrier; atomic check-and-set; leases with one requeue	Operator manipulation of the queue service	Replay the event log; fault injection
Sybil hotkeys, spam, copies	Subnet registration to join the queue; one active submission per hotkey and lineage; capacity cap; lineage similarity; exact-hash preflight; appeals	Queue occupation by many registered hotkeys; clever semantic copies	Flood and copy trials before submissions open
Malicious native code	VM boundary, reset, clean images, no secrets on workers; native code flags a winner for review	Hypervisor or driver exploits	Periodic escape tests
Operator self-dealing	Committed reports; published replay inputs; ineligible operator hotkeys; reproduction by a second evaluation group where the profile enables it	Total: evaluation is operator-run, and other validators submit the operator's scores	Reruns from published inputs; weight audits

10.5 Scoring authority

Evaluation is operator-run, and every validator submits the operator's signed weight result (Section 7.5), so the operator's scores are the outcome on chain [57]; validators other than the operator do not check performance themselves. Publishing every replay input exposes an unfair score but does not prevent it. The protections are these: every replay input is published; reports are signed and committed before they are revealed; weight results are signed, carry a validity window, are checked against a pinned key and are served only to hotkeys holding a validator permit; operator hotkeys cannot be champions; and evaluator changes are signed releases announced at least 7 days ahead with a raised weights version, applying only to submissions admitted afterwards.

11 Why RamCove is built this way

RamCove asks one question of every submission: does the same GPU now serve more requests that a customer would accept? Each design choice below follows from that question.

11.1 What is measured

Choice	What RamCove does	Why
Quantity	The highest offered rate at which every gate holds: *qualified serving capacity*	Customers buy capacity under limits, not one faster request
Load	Open-loop Poisson runs with bursts, plus a live multi-turn check (P-live)	A closed-loop or short run hides overload (Section 3.3)
Latency	Hard per-request limits; attainment overall and per context band	A fast average can hide the slow tail that users feel
Quality	Deterministic known answers, offline and under load, against a fixed reference; a length gate	Speed must never be bought with wrong or shortened answers
Memory	Evidence that the gain comes from memory (Section 8.3)	The subnet pays for using memory better, which is where the headroom lies

11.2 How a crown changes hands

Choice	What RamCove does	Why
Queue	Free to enter, continuous, ordered by the finalized block at admission, bounded; leases and a settlement barrier	Finishing fast never lets a later submission overtake an earlier one
Replacement	One reading against the champion's fixed score must clear a 5% margin; a reading close to it gets a second, and the average decides; winners reviewed before promotion	One evaluation per submission keeps the queue short; near the margin, the incumbent keeps the benefit of the doubt
Reward	King of the hill: five kings per profile; each king below King #1 earns 20%, and King #1, the champion, earns the rest, including every empty slot's share; with no king the allocation burns	Each verified win keeps paying until five newer kings displace it, the newest win earns the most, and nothing is paid for nothing
Verification	The operator-run validator scores under published rules with signed, committed reports; every validator submits the operator's signed weight result, which only validators can read; anyone can replay the published inputs	Whole-H100 isolation is costly to run, so evaluation stays with the operator; validators rely on the operator's scores

11.3 What RamCove is not

RamCove does not rent GPUs, host models for end users or train them. It is not a one-off vendor benchmark either: industry benchmarks measure capacity under latency limits for a single submission [21], while RamCove runs that measurement continuously and pays for each verified improvement. Its main alternative is the engines' own progress. When an upstream release captures a gain, the tuned baseline absorbs it and the bar rises for everyone, so RamCove is useful only while material gains remain beyond a well-tuned engine. The competition measures that, profile by profile.

11.4 Why a subnet, and what it costs

A package of verified improvements could be sold without a chain; Section 13.1 compares the routes. What a subnet adds is a standing reward budget, open to anyone, anywhere, without RamCove raising a prize fund for each improvement. Miners earn rewards by becoming champion. Their submissions must improve capacity by the required margin and pass all required checks. It is not the only way to fund an open competition, and RamCove's team also funds its own development for each profile (Section 13.1).

Its costs are concrete. Registration locked 751 TAO on 25 September 2026, about $225,000 at $299.92 [84, 85]. That TAO is not refundable and becomes the subnet's pool reserve. The subnet limit is reached, so a new subnet displaces the lowest-priced one, and a low-priced subnet can itself be pruned once its immunity ends [86]. A partner could provide an existing slot instead (Section 13.4); either way the slot is capital, not an operating cost. Pruning is enforced by the network, and RamCove cannot prevent it: a pruned subnet loses its slot (Section 13.2). The RamCove team funds launch, evaluation infrastructure and initial development using its own resources. Evaluation capacity is adjusted to handle submission demand. Customer payment is negotiated after a useful improvement has been demonstrated (Section 12.6).

12 Commercial model, costs and funding

This chapter explains where RamCove's revenue comes from, who carries the cost of checking miner submissions, who pays for each part of the work, and what that work costs. Prices marked *verified* were read on 25 September 2026. Customer examples are hypothetical. Amounts that depend on quotes or decisions are named as cost categories rather than filled with numbers.

12.1 Customer revenue and the cost of evaluation

RamCove's commercial revenue is what customers pay for demonstrated improvements and agreed services. Submitting to the competition is free: miners pay no evaluation fee, and the team carries the GPU and related cost of checking their submissions as an operating cost (Section 12.5). The subnet owner's emissions are a separate item.

Item	What it is for	When it starts	How much
Customer payments (commercial revenue)	A demonstrated improvement for the customer's own configuration, and any delivery or further services it agrees to buy (Section 4.4)	Only after a demonstration, and only if the customer agrees	Negotiated with each customer after a demonstration; no price, share, subscription or contract length is fixed in advance
Owner emissions	The chain's reward to the subnet owner, held in the treasury	When the subnet is registered	Depends on the alpha price (Section 12.8)

Each item is reported separately, and each amount is counted once, in the item it belongs to (Section 12.8). A customer request carries no fee. The team does the work of Section 4.1 steps 1 to 5 from its own resources, and revenue from a customer arrives only if step 6 produces an agreement. Customer money is paid to the operating company in ordinary currency and never passes through the chain.

12.2 What an improvement gives a customer

> **HYPOTHETICAL EXAMPLE · NOT A MEASURED RESULT**
>
> A customer serves an open model on its own or rented GPUs. After tuning, its baseline serves **10 qualified requests per second** on one GPU: answers correct, within its response-time limits, without errors. An improvement lets the same GPU and the same model serve **12 qualified requests per second**, with the same quality and response-time limits still met. That is 20% more capacity per GPU. These are round numbers for arithmetic, not a measurement and not a forecast.

The same 20% can be used in three different ways, and a customer gets whichever one fits its situation, not all three at once:

1. **Serve more demand on the same GPUs.** The fleet carries 20% more requests without new hardware. This is worth something only if the demand exists.
2. **Run fewer GPUs for the same demand.** The same demand needs 10/12 of the GPU capacity: 20% more capacity means about 17% fewer GPUs, not 20% fewer. The customer benefits only from GPUs it can actually stop running or renting. Two GPUs freed on a rented 8-GPU server still leave the whole server rented, and on owned hardware or a fixed commitment nothing is returned until the commitment ends.
3. **Postpone additional hardware.** A growing customer can serve its growth on the GPUs it has for longer before buying or renting more. The purchase is delayed, not avoided, and how long depends on how fast its demand grows.

Capacity is counted once: GPUs that are released cannot also carry extra demand. The customer's own costs also count against any benefit: its staff time, its testing, integration and deployment work, and any running cost the change adds.

No financial projections. This paper gives no customer payback, savings or return figures. Research timing is unknown, a request may find nothing, and any financial effect depends on terms negotiated after a demonstration and on the customer's own purchasing. What a demonstration shows is technical: how much more qualified work the same GPU serves while the customer's quality and response-time requirements are met.

12.3 Who pays for what, and what it costs

Team-funded does not mean cost-free. The costs below are real, and the team carries every one marked *team* before any customer pays anything. Past development spending is not counted here. The team is the founder and whoever the founder engages.

Cost	Paid by	What is known
Launch on real H100 hardware: a working evaluator, and the tuned baseline, tuning sweep, noise study and calibration for Profile 1	Team	About 150–200 GPU-hours (Section 6.4). The money depends on the hardware (Section 12.4)
Evaluation hardware with the access the isolation design needs	Team	Cost depends on owned or rented hardware and on how it is held (Section 12.4)
Engineering: evaluator operation, tuning, development of improvements, review, delivery	Team	Founder and team time; any outside specialists (Section 14) engaged as needed. Founder compensation is an open decision (Appendix E)
External security review of isolation, intake and the signed weight result	Team	Required before submissions open; amount set by quote
Legal set-up: entity, customer agreement terms and token questions	Team	Amount set by quote
Controller, storage (about 2 TB per worker-year, Section 6.3), bookkeeping, insurance	Team	Recurring; amount set by quote
A subnet slot	Team, or a slot partner (Section 13.4)	Registering directly locked 751.31 TAO at block 9,141,356, about $225,000 at $299.92 [84, 85]. The lock changes with every registration and is not refundable
The team's validator: stake, and the CPU machine that submits weights	Team	Stake amount a decision
Per request: assessment, tuned baseline, development, demonstration	Team	Depends on the request. Opening a new profile is about 150–200 GPU-hours plus engineering (Section 6.4)
Evaluating miner submissions	Team; miners pay nothing	About 1.96 GPU-hours per admitted submission, plus a reserve (Section 12.5)
A second evaluation group, only if a profile enables reproduction	Team	Hardware as for evaluation
Describing its workload; testing on its own hardware; integration and deployment	Customer	Known per request, and stated to the customer when known
Miners' development compute and time	Miners	Their own choice (Section 13.1)

Sizing the GPU cost. Two verified prices bound what a GPU-hour of evaluation can cost on rented bare metal: $6.155 when all eight GPUs of an 8-GPU node do useful work, and the whole node's $49.24 when only one of them may measure at a time (Section 12.4). Profile 1's 150–200 GPU-hours of calibration therefore cost between about $920 and $9,850 on that listed offer, before engineering, and something different on owned hardware. These are ranges, not a budget: the team sets its budget from its own resources, and Appendix E lists the amounts it decides.

12.4 Hardware and host access

The isolation design (Section 6.3) needs more than a GPU at a price. The evaluator needs **whole-GPU passthrough** to a hardware virtual machine through the host's IOMMU and VFIO driver; **host-side GPU reset** and a memory check between jobs; a **verified read-only boot image**; no other untrusted work on the measurement host; and **continuous availability** long enough for the jobs of Section 12.5. The team operates this hardware itself, owned or rented; the paper does not assume which.

Public prices on 25 September 2026, verified [43, 87, 44, 45, 51]:

Offer	Price	Access
H100 SXM in containers or virtual machines	$3.49 (RunPod) to $6.88 (AWS p5) per GPU-hour; Nebius $3.85, rising to $4.50 on 1 October	The provider controls the host. Usable for miner development; for authoritative measurement only if the security review accepts a weaker boundary (Appendix E)
Bare-metal 8-GPU H100 HGX node	$49.24 per node-hour on demand (CoreWeave), $6.155 per GPU	The whole node is rented, all eight GPUs at once

A listed price shows that an offer exists at that price; it does not establish host access. Public price lists do not state whether a provider permits VFIO passthrough, host-side resets or custom boot images, or offers the continuous availability the evaluator needs. Each host is therefore **qualified individually**: the provider confirms the access in writing, and the host passes the passthrough, reset and memory-probe checks before any of its measurements count. Renting a whole node also means paying for all eight GPUs while any one of them runs; unless the profile's calibration shows that the node's other GPUs can run candidates without disturbing the one being measured (Section 6.3), each measuring hour costs the whole node.

12.5 Evaluation scheduling and its cost

How long the jobs are. Each worker is one isolated GPU. The jobs it runs take, as planning estimates (Appendix B):

Job	GPU-hours on one worker
Screening	0.17
One evaluation	2.24 expected
A winner's extra checks	3.57
A new champion's fixed score	6.51, once per worker per promotion
The champion's answers to a new weekly bank	0.75 per worker per week

The longest job is the champion's fixed score, about 6.5 hours on each worker, and every worker must finish it before it can judge a challenger against the new champion. A worker moved to a different physical host must also be recalibrated there: the champion's fixed score measured again (another 6.51 GPU-hours) and the reset and noise checks repeated, because scores are recorded per physical GPU. Evaluation hardware is therefore held in **continuous blocks** long enough for these jobs plus the queued evaluations, not in short fixed windows, and moving between hosts has a cost. RamCove fixes no rental schedule in this paper. How the hardware is held (owned, reserved, or on demand in blocks sized to the queue) follows from which qualified hosts are available (Appendix E).

Submitting is free. Miners pay no evaluation fee. The team provides the GPU time of screening and evaluation as an operating cost, so neither the TAO price nor the number of submissions changes what a miner pays. What bounds that GPU time is the admission cap (Section 7.4): admitted work stops at two weeks of published capacity, and joining the queue needs a registered hotkey with no other active submission.

What one submission costs the team. A fully used worker holds 168 GPU-hours a week and handles about 68.6 submissions after its 20% reserve for winners' checks and new champions' scores (Appendix B), so each submission uses about 2.45 hours of worker time: about $15 at $6.155 per GPU-hour, or about $121 if each measuring hour costs a whole node's $49.24, before controller, storage and review.

Held hardware costs the same whether or not submissions arrive. At 10 submissions a week a worker is about 15% used, and the team pays for the idle remainder of its week. Evaluation capacity is adjusted to submission demand (Section 12.6), and the admission cap bounds what a burst of submissions can cost.

How evaluation is reported. Evaluation expenses (GPU time, hosting, controller and storage) are reported on their own, as the team's cost, apart from customer revenue. Customer payments are reported separately, as amounts agreed, invoiced and received on the dates each agreement sets. Customer revenue depends on agreements negotiated after demonstrations, so this paper contains no revenue figures.

12.6 Who funds RamCove

The team funds the start. The RamCove team funds launch, evaluation infrastructure and initial development using its own resources. Evaluation capacity is adjusted to handle submission demand. Customer payment is negotiated after a useful improvement has been demonstrated. The team also carries the cost of evaluating every miner submission. No customer prepayment, research deposit or outside investment is a condition of any step, and there is no fundraising target.

Requests and submission demand. Requests are scoped, prioritized or declined according to what the team can support (Section 4.2), and the admission cap limits what a burst of submissions can cost (Section 7.4).

Customer payments are revenue. A customer that agrees to pay after a demonstration buys the demonstrated improvement and the services it agreed to. Its payment is not an investment in RamCove and gives it no ownership, profit share or token.

A slot partner is a separate question. A partner may provide a subnet slot (Section 13.4). That is independent of customer payment and of the team's operating funding. RamCove does not depend on one.

Outside investment is not assumed. If the team ever sought it, the terms would be a separate decision and would not change the customer workflow of Section 4.1.

12.7 Commercial risks

- **No useful improvement for a request.** The team has carried the cost and there is no revenue from that request. This is the price of charging nothing upfront, and it is why requests are scoped and prioritized.
- **A customer declines after a demonstration.** Released code stays usable under its license, so a customer can use it without paying. RamCove earns only when finding, evidence, adaptation and delivery are worth buying (Section 4.4).
- **Capacity does not become savings.** On whole-server rental, owned hardware or fixed commitments, a real capacity gain may not reduce a bill for a long time (Section 12.2).
- **Upstream engines absorb the gain.** The tuned baseline rises and the improvement is worth less (Section 11.3).
- **The team's resources run out before revenue arrives.** Nothing in this paper forecasts when revenue starts, and the team funds the work from its own resources until then (Section 12.6).
- **Hardware access is unavailable.** If no provider offers the needed host access, the team uses its own hardware or the security review decides on a weaker boundary (Appendix E).
- **Evaluation costs outgrow the team's resources.** Evaluation is the team's cost: a long quiet period leaves held hardware idle, and a large burst fills the queue up to the admission cap.
- **Miners do not come.** If the kings' allocation is worth little at the alpha price, improvements come mainly from the team (Section 13.1).

12.8 Separate reporting

Customer revenue, evaluation expenses, protocol emissions, token holdings, proceeds of any token sale, anything a slot partner contributes, and any investor capital are reported separately. Each amount is counted once, where it belongs: evaluation expenses appear once, as the team's cost. The subnet owner's share of emissions (up to about 9,072 alpha a week on a young subnet) is worth $91, $907 or $9,072 a week at $0.01, $0.10 or $1.00 per alpha. It goes to a segregated treasury under a published policy and is not counted as customer revenue. The paper assumes no token income, no token appreciation and no sale of emissions. A large sale would not realize the spot price in any case.

13 Why Bittensor, token mechanics and governance

13.1 What Bittensor adds, and what it does not

RamCove needs a steady supply of improvements for customer-relevant profiles, each measured by the operator-run validator rather than by its author. There are several ways to buy that. Here is how they compare:

Route	How improvements are found	Cost to RamCove	Main weakness
The team alone	The team's own engineers on its own GPUs	Their time and GPU-hours, whether or not they find anything	Limited to the ideas one team can try
Contractor	Paid by the day or the project	Pay for effort, not for results	Pays for attempts that fail
Conventional bounty	One prize for one stated target	The prize, plus judging	One-off: no standing competition, and a prize fund must be raised each time

Route	How improvements are found	Cost to RamCove	Main weakness
Upstream engines	Their maintainers' roadmap	Nothing	Serves the average user; the tuned baseline absorbs it for everyone, so it is not a product
Live subnet	Anyone may submit at any time, free; five kings per profile are paid from emissions while they hold their places, King #1 the most	Evaluation, review and operation; not the reward itself	Depends on material gains existing and on serious miners taking part (Section 12.7)

What Bittensor specifically contributes. It adds a standing reward budget in the subnet's alpha token, without RamCove raising a prize fund each time. Miners earn rewards by becoming champion. Their submissions must improve capacity by the required margin and pass all required checks. It gives permissionless entry: anyone with a registered hotkey can compete. Evaluation is operator-run, and every validator submits the operator's signed weight result, which the operator serves only to validators (Section 7.5). Bittensor does not add customers, revenue, token value or protection from a free upstream. A subnet is also not the only way to fund an open competition: a company could fund prizes itself. RamCove combines two routes instead: the team funds its own development, and the subnet pays miners to beat the current reference (below).

Open source and the subnet are separate questions. Improvements are released under open licenses because customers must be able to run and audit them (Section 4.4). That would be true without a chain. The subnet is about who is paid to find them, and how they are verified.

Launch with a tuned baseline; the team and miners improve it. Each profile opens with its tuned baseline published as the version-0 reference, so launch does not wait for an improvement to be found. The team then develops and tests improvements on its own GPUs (Section 4.1). They are reference work: measured under the same rules, attributed to the team, adopted only as an announced new profile version, and never paid from emissions; operator-affiliated hotkeys cannot be champions (Section 9.3). Miners are paid only once the subnet runs in network mode and a submission beats the current champion by the margin with every gate passed. The launch requirements of Section 14.1 come before submissions open, and no miner reward is paid outside the subnet's rules.

Miners carry real costs, and most will not be paid. King of the hill pays at most five kings per profile. Every challenger pays its own development compute, and a serious attempt takes tens of GPU-hours a week. RamCove carries the whole cost of *being measured*; the cost of *building* stays with the miner. The miner-economics note below shows how thin the expected pay is for a challenger when alpha is cheap. A profile is added when a customer requests one the team can support, or when evaluation hardware and miners serve it (Section 6.4), and only when its calibration keeps verification within the limits of Section 14.1.

13.2 Chain mechanics relied on

The table lists the chain rules RamCove relies on, each with its evidence type; where documentation and chain storage differ, the paper follows chain storage. These rules changed several times in 2026, so the operator re-reads them whenever the runtime is upgraded.

Mechanism	Verified fact (evidence type)	Use in RamCove
Emission	0.5 TAO per block; a subnet's TAO share follows its alpha price through an emission gate favouring top subnets (source; docs) [57, 88]	A new low-priced subnet should expect little TAO inflow
Alpha split	Up to 1 alpha per block to participants; 18% operator, 41% miners, 41% validators and stakers (source; chain read) [57]	The kings' allocation is the miners' 41%
Operator-UID weight	Miner emission to owner-associated hotkeys is burned by default (recycle is a setting); the subnet's TAO share is scaled by one minus the burned fraction (source; chain read) [75]	No king yet, or an ineligible king; a real cost
Weights	Stake-weighted consensus clips outliers; activity cutoff 5,000 blocks at tempo 360; commit-reveal on by default (source) [89, 62]	Vector every tempo; commit-reveal enabled for RamCove
Registration and pruning	Floating burn price (0.0005–100 TAO); 256 UIDs; lowest-emission non-immune UID pruned; a new hotkey can take the same UID number; default immunity 4,096 blocks (source; chain read) [59, 78]	Rewards keyed to hotkeys; registration needed to join the queue and to be paid
Commitments	Registered hotkeys only; 128-byte raw or 512-byte fields; 3,100 bytes per hotkey per epoch; a later commitment overwrites the earlier (source) [61]	Per-tempo anchors of event log and report hashes

Mechanism	Verified fact (evidence type)	Use in RamCove
Finality	Blocks are final once the GRANDPA finality protocol confirms them (chain read) [90]	Admission keys and anchors use finalized blocks only (Section 7.2)
No slashing	No miner slash or ban; collateral enforced only by zero scores (docs; source) [79]	Disputes end in suspension or exclusion
Subnets	Registration lock 751.31 TAO at block 9,141,356 (it doubles on each registration, then decays), non-refundable and becoming the new subnet's pool reserve; the 128-subnet limit is reached, so a new registration prunes the non-immune subnet with the lowest price; immunity 864,000 blocks, about 120 days, on chain, while the documentation says six months (chain read; docs) [84, 86]	A slot is launch capital (Sections 12.3, 13.4) and can be lost to network pruning

One behaviour is exercised on the chain before submissions open on a profile: whether zeroing all miner weight could route emission elsewhere without the burn penalty.

EMISSIONS PER 7 DAYS (PRICE IS AN INPUT)

At most 50,400 alpha: operator up to 9,072; the five kings up to 20,664 together; validators and stakers up to 20,664. At $0.10 per alpha these are about $907, $2,066 and $2,066. Only the operator's validator hosts isolated whole-GPU measurement (Section 12.4); other validators need a CPU machine and stake. For scale, one listed 8-GPU bare-metal node held for a week costs about $8,272 at $49.24 an hour, four times the whole validator-and-staker share at $0.10 per alpha. Section 12 counts none of it.

ILLUSTRATIVE MINER ECONOMICS (PRICES ARE INPUTS, NOT FORECASTS)

A young subnet emits up to 1 alpha per block to participants [57]: up to 50,400 a week, of which miners get up to **20,664 alpha**, all of it to a lone King #1, and up to about **4,133 alpha** per king once five kings share it at 20% each. That is about $41, $413 or $4,133 a week per king at $0.01, $0.10 or $1.00 per alpha with five kings, and five times as much for a lone King #1, before consensus effects. Ten serious challengers each spending 60 GPU-hours a week at about $3.50, the low end of verified on-demand prices, cost $2,100 a week together, and only one of them is paid. Expected pay per challenger is therefore far below what a king earns.

13.3 Who contributes, receives, controls and risks what

Party	Contributes	Receives	Controls	Risks
Customer	A profile request: requirements and workload statistics by consent; its own testing, integration and deployment effort	Evidence of an improvement demonstrated on its configuration; under an agreement, delivery and any further work it buys; released code under its license either way	Whether to request, whether to agree after a demonstration, what to deploy	Its staff time and any testing on its own hardware
Founder and team (the operating company)	Design, code and operation; launch, assessment, baselines, development and demonstrations, on its own GPUs and from its own resources; delivery	Customer revenue under agreements; owner emissions, held in the treasury; a validator's share if it runs a validator (shared with its stakers)	The rules, evaluator releases, review, which requests to accept, customer agreements	Every cost before revenue, including the evaluation of every miner submission; work that finds nothing; customers who decline; liability for delivered code
Slot partner (optional)	A subnet slot, if the team does not register one itself	Only what a written agreement grants (Section 13.4)	Only what the agreement grants	Whatever it contributes; a slot's registration lock is non-refundable and exposed to pruning
Miners	Improvements and their own development compute; subnet registration to join the queue	While they hold their places, Kings #2 to #5 earn 20% of the profile's miner allocation each and King #1 the rest. Everyone else: nothing	Their submissions	Development cost without reward
Validators	Stake and weights: the operator's validator evaluates; the others submit its signed result or delegate stake with child hotkeys	Validator share of emissions (shared with stakers)	Their own weights	Stake cost if emissions stay small; relying on the operator's scores
Alpha holders	Stake that weights validators	Staking returns under chain rules	Nothing in the company	Token price; no revenue right, no claim on the company

An alpha purchase does not fund the company or grant revenue rights. Its TAO goes into the subnet's pool, not to the company. No buyback, burn-from-revenue or price assumption exists. The operator publishes the rules and evaluator releases. Rule changes are announced at least 7 days ahead and apply only to newly admitted submissions; on chain they take hold as validators holding most of the stake run them. Questions for counsel: obligations of holding the operator key and the treasury; liability for miner-authored code in customer contracts; exposure from exclusions; jurisdiction for emissions received by a services company; and licensing that keeps the stack maintainable if the operator ceases to exist.

13.4 A slot partner: optional, and nothing agreed

The team can register RamCove's subnet itself, or a partner could provide an existing slot while the team operates and develops RamCove. RamCove does not depend on a partner, and this paper sets no partner terms. The question is separate from customer payment and from the team's operating funding: a partner's contribution does not pay for customer work, customers pay nothing to a partner, and RamCove does not assume that a partner brings new capital.

If a partner is ever involved, a written agreement would have to settle at least:

Term	What the agreement must settle
Slot control	Who holds the subnet's operator key, and how control passes if the agreement ends
Key custody	Where the operator key and the evaluation wallet are kept, who can sign, and how many signatures a transfer needs
Operating duties	The team's obligations to run the evaluator, publish results and keep the rules; the partner's rights to reports
Contributions	What the partner contributes beyond the slot, if anything
Reporting	What figures the partner receives, using this paper's definitions (Section 12.5)
Termination	What each side keeps, what happens to the slot and to customer agreements, and what notice applies
Economic participation	Whether the partner receives anything, and of which kind below. This paper proposes no percentage

Four kinds of participation are different, and none implies another:

- **Revenue share:** a percentage of customer revenue, taken before any costs are paid.
- **Profit share:** a percentage of profit after costs, which pays nothing while costs exceed revenue.
- **Owner-reward participation:** a share of the subnet owner's alpha emissions, which otherwise go to the treasury; its value depends on the alpha price.
- **Equity:** ownership of the operating company.

RamCove promises no return to anyone who provides a slot or capital. A slot's registration lock does not come back, and a low-priced subnet can be pruned after its immunity ends (Section 13.2). Tax, securities, token and partnership questions need jurisdiction-specific professional advice before anything is signed.

14 Launch, operation and risks

RamCove grows profile by profile, starting with Profile 1: Qwen3.8-27B on the NVIDIA H100 SXM, a profile the team defined. The RamCove team funds launch, evaluation infrastructure and initial development using its own resources. Evaluation capacity is adjusted to handle submission demand. Customer payment is negotiated after a useful improvement has been demonstrated (Section 12.6).

14.1 What launch requires

Submissions open on a profile only when every requirement below holds. Launch does not require an improvement to have been found: the profile opens with its tuned baseline as the version-0 reference, and the competition starts from there.

Requirement	What must hold	Operator
Qualified measurement hosts	Each worker provides whole-GPU passthrough, host-side reset and a verified boot image (Section 6.3); a reset leaves device memory clear under the memory probe; the host is available in continuous blocks long enough for the jobs of Section 12.5	Team

Requirement	What must hold	Operator
Profile calibration	The pinned stack serves the model correctly; the tuned baseline is measured and published as version 0; reading-to-reading SD is at most 2% over ten readings on two hosts; one evaluation takes at most 5 GPU-hours; the memory diagnostics separate three known memory methods from three compute-only changes (Section 8.3)	Team
Frozen rules	δ, the clear-win line, the grid step and the admission cap are set from the calibration and published (Section 9.2)	Team
Security	An external review of isolation, intake and the signed weight result, with no open high-severity finding	Team; external reviewer
Chain	A subnet slot; admission, registration and champion-change paths exercised on chain; validator-only reads of the weight result checked against on-chain validator permits; commit-reveal weights (Section 13.2)	Team, or a slot partner for the slot

Calibration sets the rules rather than a calendar. Noise between 2% and 4% widens the margin or the second-reading band, or lowers the admission cap; noise above 4%, or an evaluation above 10 GPU-hours, keeps the profile closed until the cause is found and the calibration is repeated (Section 14.4). Two hosts run by the same team measure hardware noise, not independence (Section 10.5).

14.2 From a request to an agreement

Step	Operator	Cost category (Section 12.3)	Complete when
Profile request	Customer; team	The customer's time; the team's assessment	The request names model, GPU, workload, quality and response-time requirements; the team supports, scopes or declines it, and for a supported request tunes the customer's configuration on team hardware
Development and competition	Team; miners; operator-run validator	Team GPU time; about 150–200 GPU-hours if the request needs a new profile (Section 6.4); evaluation cost	The team develops and tests improvements; the profile is published and miners compete on it
Verified improvement	Operator-run validator; team	Evaluation	An improvement passes the evaluation rules and every gate on the profile's hardware: a champion that beats the current reference by the margin, or a team improvement adopted as an announced profile version
Demonstration	Team	Team GPU time	The customer receives evidence of a useful, reproducible improvement over its tuned baseline, and the conditions under which it holds
Negotiated agreement	Team; customer	Legal terms	A signed agreement on payment, delivery and any further services, or the customer's decision to decline, recorded

14.3 More profiles

A new profile opens when a customer requests one the team can support, or when evaluation hardware and miners serve it. It meets the same launch requirements as Profile 1: its own calibration, its tuned baseline as version 0 and its noise study published, for about 150–200 GPU-hours plus engineering (Section 6.4).

14.4 Profile maintenance and updates

A profile needs routine care for as long as it runs. The actions below are part of normal operation and follow the published rules. None of them lowers a requirement to produce a winner.

1. **When the configuration changes.** When the model revision, the engine commit, the driver or firmware, or the hardware changes, the operator publishes an updated profile version and measures its tuned baseline again. The change is announced in advance and applies only to submissions admitted after the announcement (Sections 9.2, 10.5). The champion's code is ported to the new version and its fixed score is measured again on every worker (Sections 8.8, 9.4).
2. **When measurements disagree.** When repeated measurements are inconsistent, between readings, between workers or against the calibration, the operator investigates the hardware and the evaluator, corrects the problem and repeats the calibration. The calibration thresholds stay as published: a reading-to-reading SD of at most 2% and at most 5 GPU-hours per evaluation. Between 2% and 4%, the margin or the second-reading band widens, or the admission cap falls, each through the published process for changing profile rules. Above 4%, or above 10 GPU-hours, the profile stays closed until the cause is fixed (Section 14.1).

3. **When a submission fails.** A submission that fails a correctness, quality, performance or security requirement is rejected and its reasons are published (Section 7.3). The requirements are not relaxed to let a submission through. Without a new winner, the kings keep their places and their shares (Section 9.3).
4. **When evaluation is unreliable.** If evaluation cannot produce reliable scores, for example because a worker fails its checks or its results drift, the operator pauses the affected evaluation, worker or profile while fixing it. No submission is promoted on unreliable results. Affected readings are voided and published, and admitted submissions keep their admission key: they run again with fresh tests once evaluation is reliable, under the rules for infrastructure faults (Sections 7.3, 8.8).
5. **Customer feedback.** What customers ask for, and why they accept or decline a demonstrated improvement, guides which workloads a profile uses and which profile updates come first (Section 4.2). A workload change is itself a new profile version, published and announced in the same way.

Appendix A · Profile, admission, weight-result and report formats

Formats; values illustrative; `<pinned>` values are frozen before submissions open.

```
{"profile": "ramcove-p1-h100sxm-qwen38-27b-fp8", "version": "1.0.0", "workloads": ["P-mem-v1", "P-live-v1", "P-
ctl-v1"],
 "model": {"repo": "Qwen/Qwen3.8-27B-FP8", "revision": "017b9c7af6b5689d5dd426a76e0bc077eb5ca20a",
           "shard_sha256": "<pinned x66>", "mode": "non-thinking", "input": "text", "max_model_len": 32768,
"decoding": "greedy"},
 "engine": {"repo": "vllm-project/vllm", "commit": "98dff2a81d747d1dba01a47f939f48c3526d4206",
            "image_digest": "<pinned>", "python_lock_sha256": "<pinned>", "torch": "2.13.0", "cuda": "13.0.3",
            "args_sha256": "<hash of the tuned baseline's engine arguments>",
            "allowed_paths": ["vllm/v1/core/**", "vllm/v1/attention/**", "vllm/v1/kv_offload/**"],
            "forbidden": ["vllm/model_executor/models/**", "vllm/entrypoints/**", "tests/**"]},
 "hardware": {"gpu": "1x NVIDIA H100 SXM 80GB, whole device, no MIG", "driver": "<pinned>", "firmware":
"<pinned>",
              "power_limit_w": "<SXM default, checked per run>", "gpu_ceiling": "device memory - 1.5 GiB, driver-
sampled",
              "tight_ceiling": "ceiling - 25% of champion cache budget", "cpu": "16 dedicated cores, <model>, GPU
NUMA node",
              "vm_ram_gib": 64, "swap": false, "network": "none", "isolation": "hardware VM, VFIO whole-GPU
passthrough"},
 "gates": {"ttft_s": "1.0 + 0.1/1k prompt tokens", "tpot_mean_ms": 100, "stall_s": 2.0, "deadline_s": 60,
           "attainment": 0.95, "band_attainment": 0.90, "errors": 0.005, "drift_q4_over_q2": 1.40,
           "burst_recovery": 1.35, "stability_floor": "p90 at least 50% of its own limit",
           "drain": "nothing in flight at the end", "length_ratio_vs_reference": 0.90,
           "quality_pp": {"offline": -2.0, "stratum": -3.0, "vs_reference": -3.0, "bound": "one-sided 97.5%",
                          "floors": {"overall": 85, "general": 85, "per_position": 80}},
           "accuracy_under_load_pp": {"estimate": -2.0, "bound": -3.0},
           "memory_evidence": ["tight_budget_vs_fixed_score", "external_accounting", "declared_mechanism"]},
 "replacement": {"delta": 0.05, "clear_win": 0.09, "second_reading": "if delta < G < clear_win; mean must exceed
delta",
                 "grid_step": 0.03, "grid_steps": [-8, 16], "champion_score": "once per worker, when crowned",
                 "beacon_min_delay_min": 30, "second_reading_round_after_h": 2,
                 "reproduction": "one reading by a second evaluation group, only if reproduce_winners is set (off
by default)"},
 "kings": {"slots": 5, "king_1": "the current champion: its own 20% plus every empty slot's share",
           "kings_2_to_5": "20% each of the profile's miner allocation",
           "new_king": "becomes King #1; the others move down one place; King #5 leaves",
           "no_king_or_ineligible_king": "operator UID, burned"},
 "queue": {"one_active_per": ["hotkey", "lineage"], "daily_limit": null, "admission_cap": "2 weeks of booked
work",
           "submission_fee": null, "registered_hotkey_to_join": true, "booked_per_submission_gpu_h": 2.41,
           "admission_key": "[finalized block at admission, index within it]", "invitation_h": 24,
           "leases_h": {"screen": 2, "evaluation": 24, "reproduction": 8}},
 "question_bank": {"period_days": 7, "epoch": "2026-09-21T00:00:00Z", "screening_items": 200,
                   "quality_items": 4000, "commit_ahead_h": 24, "seed": "HMAC(operator secret, profile, period)"},
 "weight_result": {"valid_tempos": 2, "read_by": "hotkeys with a validator permit",
                   "read_signature": "sr25519 over hotkey, method, path, timestamp", "clock_window_s": 120},
 "evaluator": {"version": "0.1.0", "weights_version_key": 100}}
```

```
{"admission": "adm-000481", "submission": "sub-000481", "hotkey": "5F...", "registered_uid": 37,
 "artifact_sha256": "9c1e...", "profile": "ramcove-p1@1.0.0", "admission_key": [9200042, 3],
 "booked_gpu_h": 2.41, "lineage": "lin-7a2", "state": "admitted", "signature_intake": "<intake signature>"}
{"schema": "ramcove.weight_document/1", "network": "finney", "netuid": "<subnet>", "issued_block": 9200400,
 "valid_until_block": 9201120, "burn_hotkey": "<subnet owner>", "shares": [
   {"hotkey": "5K1...", "share_bp": 8000, "role": "king"}, {"hotkey": "5K2...", "share_bp": 2000, "role":
"king"}],
 "signature_owner": "<sr25519>"}
{"read": "GET /v1/weights/latest", "x-ramcove-hotkey": "5V...", "x-ramcove-timestamp": 1790000000,
 "x-ramcove-signature": "<sr25519 over ramcove/v1|weight_fetch|network|hotkey, method, path, timestamp>"}
```

```
{"comparison": "cmp-000481", "champion": "champ-v1 (sha256:41d0...)", "challenger": "sha256:9c1e...",
 "worker": "w-02", "validator": "5D...", "anchor_block": 9203610, "drand_round": 28751440,
 "seed": "sha256(drand_randomness || comparison id)", "champion_fixed_score": 2.40,
 "reading": {"grid_steps_passed": 2, "rate": 2.5462, "gain": 0.0609, "runs": 4},
 "decision": "another_reading", "gates": {"all_runs_logged": true, "voided": false}, "gpu_hours": 1.4,
 "report_sha256": "b7c2...", "signature_validator": "<sr25519>"}
```


Appendix B · Workload, statistics, parameters, queue and costs

Generator. The week's bank seed, mixed with each reading's own seed, fills public templates with a persona, one to three synthetic documents, a fact table (planted facts, distractors, token positions) and a turn script. Graded questions (lookup, relational, negation) target the early, middle and late thirds. In P-mem, assistant turns are slot-filled text carrying facts; in P-live they are the model's actual answers. Turn k in P-mem is issued at conversation start plus earlier turns' 8 s service allowances and lognormal think times, on schedule even if the previous turn is late. Prompt plus maximum output never exceeds the context limit.

Memory pressure (illustrative, λ = 2.40 req/s). Mean turns 2.57 → 0.93 new conversations/s; mean think time 30·e^0.18 ≈ 35.9 s → lifetime ≈ 1.57 × 43.9 + 8 ≈ 77 s → about 72 open conversations holding about 580,000 tokens: 35.4 GiB of attention cache at 64 KiB per token plus 72 × 74.8 MiB = 5.3 GiB of recurrent state, about 40.7 GiB against a 40 GiB budget.

Run procedure. Per job: fresh VM from a verified image; driver, error-correcting memory (ECC), clock and power checks; canary microbenchmark. Per run: engine start (about 4 min, budgeted), 8-minute warmup with separate-seed conversations, 20-minute window (8 for each rate of a reading) with bursts, drain up to 90 s, gates, signature.

Quality statistics. For paired binary outcomes with discordance d, SE ≈ $\sqrt{(d/n)}$; bound = observed difference − 1.96·SE [68].

Scope	Items	SE (d = 10%)	Bound if equal	Bound at −0.6	Margin
Offline overall	4,000	0.50	−0.98	−1.58	−2.0
Position stratum	≈ 1,067	0.97	−1.90	−2.50	−3.0
Context band	800	1.12	−2.19	−2.79	−3.0
Under load, pooled, unpaired (p ≈ 0.91): a winner's 1-hour soak against the champion's three confirmation runs	≈ 2,500 vs 2,780	0.79	−1.55	−2.15	−3.0 (point −2.0)

Replacement statistics. A reading steps up a grid spaced 3% apart (ln 1.03 = 0.0296) from the champion's fixed score until the first rate that fails. A first reading at or below 5% ends the evaluation and one at or above 9% wins. A reading in between, which on this grid means +6.1%, gets one more reading, and the average of the two must exceed 5%. In a simple model in which each run's edge varies with 2% noise, a true 3% gain wins about 0.1% of the time, 5% about 7%, 8% about 69% and 10% about 95%. The planning rate for first readings that need the second is 15%. The fixed score is confirmed with three fresh runs, so its own noise is smaller than a single reading's; each profile's calibration measures both.

Key parameters (defaults, set by each profile's calibration unless fixed).

Parameter	Default	Rationale
GPU ceiling; tight budget	Device − 1.5 GiB; minus 25% of cache budget	Tuned baseline uses the same ceiling; scarcity for memory evidence
Candidate VM	16 dedicated cores, 64 GiB, no swap, no network	No unlimited offload or contention; not host sizing
TTFT; TPOT; stall; end-to-end	1.0 s + 0.1 s/1k tokens; 100 ms mean; 2.0 s; TTFT + 0.15 s/token, ≤ 60 s	Baseline must pass at 80% capacity
Replacement	δ 5%; clear win 9%; one more reading in between, mean above δ; grid 3%, up to +60%	Above noise; a true 10% gain wins ≥ 80%
Queue	Free; one active per hotkey and lineage, no daily limit; a registered hotkey to join; cap 2 weeks of booked work; 2.41 GPU-hours booked per admitted submission; leases 2/24/8 h	Bounded backlog; no stuck jobs; no submission window
Question bank	Operator platform, every 7 days; 200 screening and 4,000 quality questions plus traffic scripts; fingerprint a day ahead, seed published after the week	Validators neither write nor choose questions; every bank can be checked afterwards

Parameter	Default	Rationale
Submission	Free; the operator provides the evaluation GPU time	A miner's work reaches measurement through the free checks, registration and the queue alone
Champion's score	Measured once per worker when crowned; not measured again within a profile version	Every challenger meets the same number; no daily or weekly re-measurement
Registration	A winner is promoted only once its hotkey is registered, within 14 days; a champion that deregisters has 14 days to return	Registration needed to join the queue; pay follows the hotkey
Hyperparameters	Tempo 360; commit-reveal on; min weights 1; integer weights version; immunity long enough for a new champion's first weights (it cannot cover 14 days); meaningful minimum burn	Per-tempo vectors; costly registration floods

Queue and cost model (planning estimates)

Stage	GPU-hours	Basis
Screening	0.17	Engine start, 200 bank questions, memory check
One evaluation	2.24 expected	The challenger's answers to the 4,000 bank questions 0.75; GPU checks and re-image 0.25; one reading of about 3 rates × 0.36 h; a second reading for the 15% of first readings close to the margin
Booked per admitted submission	2.41	Screening plus one expected evaluation, held until the result is known
Winner checks	3.57	Tighter-budget reading (a second for the 25% that miss the margin) and accounting run 1.70; P-ctl 0.31; P-live 0.56; soak 1.00
Champion's fixed score	6.51 per worker, once per promotion	Search 2.15; confirmation 2.18; tighter budget 1.12; P-ctl 0.31; answers to the current bank 0.75
Champion's answers to a new bank	0.75 per worker per week	The 4,000 questions of each week's bank, answered once
Per admitted submission (used)	**1.96 expected**	0.17 + 0.8 × 2.24 (screen pass 80%: a planning rate)
Worker capacity	134.4 job-hours → 68.6 submissions per week	168 h − 20% reserve: winners' checks (2.7 winners a week at a 5% planning rate, 9.8 h), a new champion's score (6.5 h), the weekly bank answers (0.75 h), downtime and reruns

A *worker* in these tables is one GPU available all week. Hardware held for part of the week handles proportionally fewer submissions, and each block it is held for must be long enough for a new champion's 6.51-hour fixed score (Section 12.5). *Arithmetic illustration only:* 100 evaluations of about 2.2 hours need about 220 GPU-hours, which is 220 wall-clock hours on one worker, 55 on four and 22 on ten if perfectly distributed. Screening, winners' checks, new champions' scores, reproduction, re-imaging, review and downtime come on top, as below.

Demand	1 worker	4 workers	10 workers
Burst of 10 (20 GPU-h)	≈ 24 h (1.0 day)	≈ 6 h, but no job finishes before about 2.4 h	≈ 2.4 h, floored by the longest single job
Burst of 100 (196 GPU-h)	≈ 245 h (10.2 days); cap 111	≈ 61 h (2.6 days); cap 446	≈ 24 h (1.0 day)
Sustained 10 per week	15% utilization	4%	1%
Sustained 100 per week	Capacity 69: backlog grows 31 per week; waiting list absorbs it	Capacity 274: 36% utilization	Capacity 686: 15%

Human review of a flagged winner (up to 10 business days) and weight publication add elapsed time after the winner checks. If arrivals exceed completion capacity, the queue grows, and the cap diverts the excess to the free waiting list. **Cost:** each admitted submission uses about 2.45 hours of a fully used worker's time (168 h ÷ 68.6). At the listed bare-metal price of $6.155 per GPU-hour (an 8-GPU H100 node at $49.24 per hour, verified 25 September 2026 [51]) that is about $15.07. If only one GPU of a rented node may measure at a time (Section 6.3), each measuring hour costs the whole node and a submission about $120.59. Both figures come before controller, storage and review, and the operator carries them: miners pay nothing to be evaluated. A host counts only once it is qualified (Section 12.4). Below full use, the team pays for the idle hours (Section 12.5).

Appendix C · Pseudocode

Pseudocode only; not implementation code and never run. All state changes append to the signed event log.

```
preflight(sub):                                     # public, free, executes nothing
    require size_ok and manifest_ok and sig_ok(sub.hotkey) and profile_open(sub.profile)
    require paths_allowed and licenses_declared and not excluded(sub.hotkey)
    require sha256(sub.artifact) not in all_submissions and ≠ champion.hash
    require no_active(sub.hotkey) and no_active(lineage_of(sub))
    return admit_or_waitlist(sub)                   # waitlist ordered by signed preflight time

admit_or_waitlist(sub):                             # free: no quote and no payment
    require registered(sub.hotkey)                  # joining the queue needs a registered hotkey
    if booked_hours + BOOKED_PER_SUBMISSION > cap_hours(profile): return waitlist.add(sub)   # 2 weeks
    head = finalized_head()
    key = (head, admissions_in(head))               # finalized block, index within it
    booked_hours += BOOKED_PER_SUBMISSION           # 2.41 GPU-hours of the operator's time
    return sign(admission(sub, admission_key = key))    # a place in line nobody can buy or change

worker_loop(w):
    champ = champion.current();  bank = operator.bank_for(week_now())   # the week's seed, never the secret
    if w.score_version ≠ champ.version:             # once per champion, never again
        w.score = fixed_score(champ, w)             # search + 3 confirming runs; tighter budget; P-ctl
    if w.answers.bank ≠ bank.id: w.answers = answers(champ, bank)    # once per worker per weekly bank
    job = queue.lease(oldest admission_key where compatible(w), lease = LEASE[job.stage])
    match job.stage:                                # champion version frozen for this job
      screen:  r = screen(job)                      # build VM, start, 200 items, contract, memory
               if decisive_failure(r): finish(job, "rejected", report = r)
               else: queue.push(job, stage = evaluation)
      evaluation:
               if not quality_ok(job, bank, w.answers): finish(job, "not_promoted"); return
               g = reading(job, w.score)            # step up the 3% grid until a rate fails
               if DELTA < g < CLEAR_WIN: g = (g + reading(job, w.score)) / 2    # one more reading
               if g ⩽ DELTA or not winner_checks(job, w.score):  # memory, P-ctl, P-live, soak
                   finish(job, "not_promoted"); return
               job.gain, job.champ = g, champ.version
               if profile.reproduce_winners: queue.push(job, stage = reproduction)   # another evaluation group
               elif flags(job): open_review(job)    # native code, third-party code, scans, disputes
               else: job.state = "ready"
    on lease expiry: requeue once at same key; second expiry → finish(job, "operator_failed")

reading(job, score):
    rec = record(job, drand_round = first round ⩾ anchor_final + 30 min);  anchor(rec)
    seed = H(drand(rec.round) || rec.id);  k = 0
    while k ⩽ GRID_MAX and run_passes(job, rate = score * 1.03^k, seed): k += 1
    if k == 0: return -1                            # did not reach the champion's score
    return 1.03^(k - 1) - 1                         # infra fault → void(rec), new record, new round

settle():                                           # run after every job event
    for job in ready ordered by admission_key:
        if earlier_unfinished(job): break           # settlement barrier
        if job.champ ≠ champion.current():          # another winner was crowned first
            if rules_changed(job) or rolled_back(job): requeue(job, same key); continue
            job.gain = (1 + job.gain) / product(1 + g for g in gains_crowned_since(job.champ)) - 1
            if job.gain ⩽ DELTA: finish(job, "not_promoted"); continue   # nothing measured again
            job.champ = champion.current()
        if not registered(job.hotkey):              # a crown must go to a hotkey that can be paid
            if waited(job) > REGISTRATION_GRACE: finish(job, "expired_unregistered"); continue
            break                                   # keeps its place; nothing measured again
        atomically: if champion.current() == job.champ: champion.set(job)   # compare-and-set

weights(tempo):                                     # the operator signs this; only validators may read it
    kings = king_queue(profile)                     # King #1 = current champion, then the kings it displaced
    share = {slot: (0.2 if slot ⩽ len(kings) else 0.0) for slot in 2..5}
    share[1] = 1.0 - sum(share.values())            # King #1 also holds every empty slot's share
    w = {owner_uid: 0.0}
    for slot in 1..5:
        k = kings[slot] if slot ⩽ len(kings) else None
        if k and eligible(k) and registered(k.hotkey): w[uid_of(k.hotkey)] += share[slot]
        else: w[owner_uid] += share[slot]           # no king, or an unpaid king: burned by chain rule
    return sign_owner(result(w by hotkey, valid_for = 2 tempos))

validator_cycle(v):                                 # every validator, the operator's included; CPU only
    r = get("/v1/weights/latest", signed_by = v.hotkey, at = now())   # served only with a validator permit
    require verify(r, PINNED_OWNER_KEY) and r.chain == chain() and r.valid_at(finalized_head())
    commit_reveal_set_weights(uids_from_own_chain_read(r), version_key = profile.weights_version)
```

Appendix D · Claim and evidence ledger

Evidence types: **doc** (published documentation), **code** (inspected source at a named commit), **chain** (finalized-block read with the query script, block number, hash and raw response retained), **API** (read-only operator API response, retained), **author-reported** (published result not reproduced), **design**, **estimate**, **illustrative**, **founder-reported** (stated by the founder, not independently checked) and **proposed** (a design or commercial term that takes effect only when agreed). Each source's access date is listed with it under References.

Claim	Type	Source	Status
Qwen3.8-27B-FP8 revision `017b9c7a`, Apache-2.0, 30,866,866,928 weight bytes; 16 attention layers at 64 KiB/token (16-bit) plus 48 recurrent layers; served by vLLM from 0.17.0	API; doc; code	[37, 1, 39, 40]	verified
vLLM v0.29.0 tag = commit `98dff2a8`; torch 2.13.0; CUDA 13.0 default build; patch releases exist	API; code	[38]	verified
H100 SXM vs PCIe: 700 W vs 350 W; 3 vs 2 TB/s; PCIe ≈ 65% of SXM performance	doc	[41, 42]	verified (vendor claim)
Container Toolkit escapes through 2026 (fixed 1.19.1); gVisor weak against driver bugs; CUDA memory not cleared	doc	[47, 48, 49]	verified; whether a reset clears device memory is checked by the memory probe on every host (Section 6.3)
H100 SXM on demand $3.49–$6.88 per GPU-hour (RunPod container, Nebius, Lambda and AWS virtual machines); bare metal only as an 8-GPU node at $49.24 per hour (CoreWeave); no committed bare-metal rate published	doc; API	[43, 87, 44, 45, 51]	prices verified 2026-09-25; access conditions are not in public price lists, so each host is qualified before use (Section 12.4)
8-bit KV cut cache memory to ~54%; eviction fails on multi-turn tasks; capacity under latency limits, goodput, open-loop load	author-reported; doc	[10, 18, 21, 22]	as published
Emission 0.5 TAO/block; 18/41/41 split	code v459; chain	[88, 77]	verified
Operator-hotkey miner emission burned by default; TAO share scaled by (1 – burned)	code v459	[75]	verified; changed twice in Aug 2026
Immunity default 4,096 blocks, operator setter limited only by u16 (max 65,535 ≈ 9 days); new hotkey reuses UID number	code; chain	[78, 77]	verified
Commit-reveal default on; activity cutoff 5,000; commitments 3,100 bytes/epoch, later overwrites	code; chain	[61, 77]	verified
Blocks final once GRANDPA confirms them; admission keys use the finalized head	chain	[90]	verified in chain reads
Registration lock 751.31 TAO at block 9,141,356; 128-subnet limit reached; immunity 864,000 blocks on chain (documentation: six months); TAO $299.92	chain; API; doc	[84, 86, 85]	verified 2026-09-25; chain and documentation disagree on immunity, so the chain value is used
No miner slashing	doc; code	[79, 80]	verified
Queue, admission, replacement, reproduction, review, weight-result access and isolation rules	design	Sections 6–9	specified
Runtimes (2.2, 3.6, 6.5 GPU-hours), pass rates, demand, prices, margins	estimate	Appendix B; Section 12	planning values; set by each profile's calibration
Customer benefit example (10 → 12 qualified requests per second), memory example, champion examples, emissions figures	illustrative	Sections 4, 8, 9, 12, 13	hypothetical inputs, not measurements; arithmetic exact
Customer workflow: launch with a tested baseline; a profile request with no upfront fee; team-funded development and the miner competition; a verified improvement and a demonstration; terms negotiated only afterwards	design (founder's rule)	Sections 4.1–4.2, 12, 14	terms negotiated with each customer after a demonstration
Profile 1 was defined by the team	founder-reported	Section 4.1	stated by the founder
Cost categories and GPU-hour ranges	design; estimate	Sections 6.4, 12.3; Appendix B	GPU-hours are planning values; other amounts are set by quotes
Submitting is free; the team carries the cost of evaluation, reported apart from customer revenue	design (founder's rule)	Sections 5.3, 7.3, 12.1, 12.5, 12.8	applies from the opening of submissions

Claim	Type	Source	Status
King #1 earns its own 20% and every empty slot's share; Kings #2 to #5 earn 20% each; with no king, the allocation burns	design (founder's rule)	Sections 9.1, 9.3	specified
Operator-run evaluation; validators submit the operator's signed weight result, which only hotkeys with a validator permit can read	design	Sections 7.5, 10.5	other validators rely on the operator's scores and do not re-run evaluations
Slot partner: optional; the terms a written agreement would need	proposed	Section 13.4	optional; this paper sets no terms

Appendix E · Open policy and commercial decisions

1. **Commercial pricing.** How payment, delivery and further services are priced after a demonstration, negotiated with each customer. Nothing is fixed in this paper (Sections 4.2, 12.1).
2. **The subnet slot.** Registration by the team, or a slot partner. If a partner: every term of Section 13.4, including whether any revenue share, profit share, owner-reward participation or equity is granted, with jurisdiction-specific advice.
3. How profile requests are assessed and prioritized against the team's resources, and when one is scoped down or declined (Section 4.2).
4. What a demonstration contains: the hardware, limits, quality evidence and reproduction method, and whether any of it runs on the customer's own hardware (Section 4.1).
5. How a team baseline update, adopted as a new profile version, treats a reigning miner champion (Sections 4.3, 9.4).
6. Whether a profile version enables reproduction by a second evaluation group (Section 7.5).
7. How evaluation hardware is held: owned, reserved, or on demand in blocks sized to the queue; and whether the security review would accept virtual machines with a weaker boundary (Sections 12.4, 12.5).
8. Budget amounts once quoted: the security review, legal set-up, hardware, and a second evaluation group if reproduction is enabled (Section 12.3).
9. The founder's compensation, and when it is paid.
10. The stake of a validator run by the team, if it runs one.
11. Legal structure.
12. Review criteria and the automatic checks that flag a winner for review, the reviewer pool, appeals, similarity thresholds, and the quality margins frozen with each profile.
13. Custody of the operator's question-bank secret, and how the evaluation workers hold a week's bank so that a leak is detectable.
14. Order of the next profiles: which model families and GPU types open after the first (Section 6.4).

Appendix F · Glossary

Admission: a submission's free entry into the queue, for a registered hotkey when a slot is free; recorded in a signed admission record. **Admission key:** the finalized block number at a submission's admission and its index among that block's admissions; sets queue order. **Alpha / TAO:** a subnet's token / Bittensor's base token. **Anchor:** an on-chain commitment of a hash of the event log or a report. **Champion:** King #1, the approved submission every challenger must beat. **Cosmetic copy:** the same method disguised by renaming or reformatting. **Demonstration:** evidence given to a customer of a verified, reproducible improvement over its tuned baseline, with the hardware, limits, quality results and configuration it holds under; commercial terms are discussed only after it. **drand:** a public randomness beacon with numbered future rounds. **Fixed score:** the champion's qualified serving capacity on one worker, confirmed once when it is crowned and not measured again; every challenger on that worker is compared with it. **Flagged:** a submission an automatic check, a dispute or a customer requirement marks for closer scrutiny. **GPU fleet:** a customer's collection of GPUs; there is no minimum size. **Hotkey / UID:** a participant's signing key / a numbered slot on a subnet that another hotkey can later occupy. **King / king queue:** one of up to five places per profile. Kings #2 to #5 each earn 20% of the profile's miner allocation, and King #1, the champion, earns the rest, including every empty slot's share; a new champion becomes King #1, the others move down one place, and King #5 leaves the queue. **King of the hill:** RamCove's

reward rule: the queue of five kings above; with no king, and for an ineligible king, the share is burned. **KV cache:** per-token keys and values kept in GPU memory so later tokens can attend to them. **Lease:** a time-limited claim by a worker on one job. **Lineage:** a submission and its revisions. **MIG:** NVIDIA's partitioning of one GPU into smaller instances; not used for authoritative runs. **Network mode:** the subnet in live operation: candidates measured in hardware VMs on whole H100 SXM GPUs, and registrations, weights and anchors on the chain. **Operator:** the operating company that runs intake and releases, and delivers customer work under agreements. **P-mem / P-live / P-ctl:** memory-bound multi-turn profile / check using actual model answers / short-request control. **Preflight:** free checks before admission that execute nothing. **Profile:** the frozen model, engine, hardware, workload and rules defining one leaderboard; RamCove adds profiles over time, and the first is Qwen3.8-27B on the H100 SXM. **Profile request:** a customer's description of the model, GPU type, workload, quality and response-time requirements it wants improved; it carries no fee. **Qualified serving capacity:** highest offered rate at which all gates hold. **Question bank:** the operator platform's secret questions and traffic scripts for one week; its fingerprint is published before the week and its seed after it. **Re-compare:** setting a waiting winner's measured gain against a champion crowned after its evaluation, by arithmetic, without measuring again. **Reading:** one capacity measurement of a challenger: single runs stepping up a 3% grid from the champion's fixed score until a rate fails. **Recurrent state:** the fixed-size memory a linear-attention (Gated DeltaNet) layer keeps for each conversation instead of a per-token cache; Qwen3.8-27B keeps it in 48 of its 64 layers. **Reference stack:** tuned engine plus the current champion's code. **Reference work:** improvements the team develops for a profile, measured under the same rules, published with its attribution and adopted only as an announced new profile version; they earn no miner reward. **Revenue share / profit share:** a percentage of revenue before any costs / a percentage of profit after costs. **Settlement barrier:** rule that results settle in admission order. **Slot partner:** a party that might provide an existing subnet slot under a written agreement; optional, and none is secured. **SXM:** the H100 module form used on HGX baseboards, distinct from the PCIe card. **Tempo:** a subnet's consensus period, 360 blocks by default [58]. **TTFT / TPOT:** time to first content token / mean time per output token. **Tuned baseline:** a configuration's engine and model, tuned by the team on team-controlled hardware; each profile launches with its tuned baseline as the version-0 reference, and a customer's improvement is measured against its own tuned baseline, not an untuned configuration. **Validator:** a registered hotkey that sets weights. The operator-run validator runs miner submissions on the profile's specified hardware, checks correctness and performance, and signs the scores; other validators submit the operator's signed weight result. **Waiting list:** free queue for interest beyond published capacity. **Weight result:** the operator's signed list of each hotkey's share of the miner allocation, valid for two tempos and renewed every tempo; served only to requests signed by a hotkey with a validator permit; every validator checks it and submits it as its weights. **Worker:** one isolated H100 host that runs evaluation jobs.

References

Numbered in order of first citation. Unless marked otherwise, each source was read on its primary site; dates are access dates unless a version is named. *partial* = secondary or ambiguous source.

[1] Qwen/Qwen3.8-27B config.json and model card: 48 Gated DeltaNet.... repo created 2026-08-05; accessed 2026-09-17. https://huggingface.co/Qwen/Qwen3.8-27B/raw/main/config.json.

[2] SGLang cookbook for Qwen3.8-27B: hybrid memory split.... v0.5.19; accessed 2026-09-17. https://docs.sglang.io/cookbook/autoregressive/Qwen/Qwen3.8-27B.

[3] Efficient Memory Management for Large Language Model Serving.... 2023-09-12. https://arxiv.org/abs/2309.06180.

[4] vLLM Optimization and Tuning: preemption is recompute in V1.... v0.29.0; accessed 2026-09-17. https://docs.vllm.ai/en/latest/configuration/optimization.html.

[5] Qwen3.8-27B-FP8 model card: most capable generation of Qwen's.... accessed 2026-09-19. https://huggingface.co/Qwen/Qwen3.8-27B-FP8.

[6] vLLM Automatic Prefix Caching design: block hashing, LRU.... v0.29.0; accessed 2026-09-17. https://docs.vllm.ai/en/latest/design/prefix_caching.html.

[7] SGLang HiCache System Design: GPU (L1), host memory (L2).... accessed 2026-09-17. https://docs.sglang.io/docs/advanced_features/hicache_design.md.

[8] vllm/config/cache.py at v0.29.0: gpu_memory_utilization default.... tag v0.29.0; accessed 2026-09-17. https://raw.githubusercontent.com/vllm-project/vllm/v0.29.0/vllm/config/cache.py.

[9] vLLM KV Offloading Usage Guide: OffloadingConnector, CPU tier.... v0.29.0; accessed 2026-09-17. https://docs.vllm.ai/en/latest/features/kv_offloading_usage/.

[10] The State of FP8 KV-Cache and Attention Quantization in vLLM.... 2026-04-22. https://vllm.ai/blog/2026-04-22-fp8-kvcache. *partial*.

[11] H2O: Heavy-Hitter Oracle for Efficient Generative Inference of.... 2023-06-24. https://arxiv.org/abs/2306.14048.

[12] SnapKV: LLM Knows What You are Looking for Before Generation.... 2024-04-22. https://arxiv.org/abs/2404.14469.

[13] KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.... 2024-02-05. https://arxiv.org/abs/2402.02750.

[14] KVQuant: Towards 10 Million Context Length LLM Inference with.... 2024-01-31. https://arxiv.org/abs/2401.18079.

[15] Quest: Query-Aware Sparsity for Efficient Long-Context LLM.... 2024-06-16. https://arxiv.org/abs/2406.10774.

[16] DuoAttention: Efficient Long-Context LLM Inference with.... 2024-10-14. https://arxiv.org/abs/2410.10819.

[17] KV Cache Compression, But What Must We Give in Return? A.... 2024-07-01. https://arxiv.org/abs/2407.01527.

[18] SCBench: A KV Cache-Centric Analysis of Long-Context Methods.... 2024-12-13. https://arxiv.org/abs/2412.10319.

[19] The Sparse Frontier: Sparse Attention Trade-offs in Transformer.... 2025-04-24 (v1). https://arxiv.org/abs/2504.17768v1.

[20] RDKV: Rate-Distortion Bit Allocation for Joint Eviction and.... 2026-05-08. https://arxiv.org/html/2605.08317v1.

[21] MLPerf Inference Rules: Server scenario with Poisson arrivals.... master at commit d3eba2f, 2026-08-20; accessed 2026-09-17. https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc.

[22] DistServe: Disaggregating Prefill and Decoding for.... 2024-01-18 (v3 2024-06-06). https://arxiv.org/abs/2401.09670.

[23] Open Versus Closed: A Cautionary Tale (Schroeder, Wierman.... 2006. https://www.usenix.org/legacy/event/nsdi06/tech/full_papers/schroeder/schroeder.pdf.

[24] vLLM v0.29.0 release (GitHub releases API; PyPI 0.29.0.... 2026-09-09. https://github.com/vllm-project/vllm/releases/tag/v0.29.0.

[25] SGLang v0.5.19 release (Apache-2.0); unified radix tree default. 2026-09-05. https://github.com/sgl-project/sglang/releases/tag/v0.5.19.

[26] vllm bench serve (vllm/benchmarks/serve.py): --request-rate.... tag v0.29.0; accessed 2026-09-17. https://raw.githubusercontent.com/vllm-project/vllm/v0.29.0/vllm/benchmarks/serve.py.

[27] AIPerf README: concurrency, request-rate, trace-replay modes.... v0.12.0, 2026-08-06; accessed 2026-09-17. https://github.com/ai-dynamo/aiperf.

[28] drand Timelock Encryption documentation. accessed 2026-09-17. https://docs.drand.love/docs/timelock-encryption/.

[29] Bittensor Docs, Timelock: timelocked commitments revealed.... accessed 2026-09-17 (references runtime v446). https://www.bittensor.com/docs/guides/timelock.

[30] vLLM Quantized KV Cache documentation: default scales 1.0.... v0.29.0; accessed 2026-09-17. https://docs.vllm.ai/en/latest/features/quantization/quantized_kvcache.html.

[31] PyramidKV: Dynamic KV Cache Compression based on Pyramidal.... 2024-06-04. https://arxiv.org/abs/2406.02069.

[32] EpiCache: Episodic KV Cache Management for Long-Term.... 2025-09-22. https://arxiv.org/abs/2509.17396.

[33] Tangram: Unlocking Non-Uniform KV Cache Compression for.... 2026-06-04. https://arxiv.org/abs/2606.06302.

[34] ShadowKV: KV Cache in Shadows for High-Throughput Long-Context.... 2024-10-28. https://arxiv.org/abs/2410.21465.

[35] LMCache README: tiered KV offload, CacheBlend, vLLM and SGLang.... dev branch; accessed 2026-09-17. https://github.com/LMCache/LMCache.

[36] vLLM tracking issue #26201, Prefix Caching for Hybrid Models.... opened 2025-10-03; open on 2026-09-19. https://github.com/vllm-project/vllm/issues/26201.

[37] Hugging Face API, Qwen/Qwen3.8-27B-FP8: revision.... repo created 2026-08-13, last modified 2026-08-14; accessed 2026-09-19 (API read, retained). https://huggingface.co/api/models/Qwen/Qwen3.8-27B-FP8.

[38] vLLM v0.29.0 release and tag ref: commit.... released 2026-09-09; accessed 2026-09-17. https://github.com/vllm-project/vllm/releases/tag/v0.29.0.

[39] vLLM recipe for Qwen3.8-27B: vLLM 0.17.0 or later; single-GPU.... accessed 2026-09-19. https://recipes.vllm.ai/Qwen/Qwen3.8-27B.

[40] vllm/model_executor/models/registry.py at v0.29.0.... tag v0.29.0; accessed 2026-09-19. https://raw.githubusercontent.com/vllm-project/vllm/v0.29.0/vllm/model_executor/models/registry.py.

[41] NVIDIA H100 Tensor Core GPU datasheet: H100 SXM vs H100 PCIe.... accessed 2026-09-17. https://www.nvidia.com/en-us/data-center/h100/.

[42] NVIDIA H100 architecture guide v1.04: SXM5 80GB HBM3, PCIe 80GB.... accessed 2026-09-17. https://www.nvidia.com/en-us/data-center/h100/.

[43] RunPod pricing: H100 SXM $3.49 Secure / $2.69 Community per.... page updated 2026-09-13; re-read 2026-09-25. https://www.runpod.io/pricing.

[44] Lambda pricing: H100 SXM on demand 1x $4.29, 2x $4.19, 4x.... re-read 2026-09-25. https://lambda.ai/pricing.

[45] AWS Price List API, us-east-1: p5.48xlarge $55.04 per hour on.... published 2026-09-24; read 2026-09-25. https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/us-east-1/index.csv.

[46] CVE-2024-0132: NVIDIA Container Toolkit 1.16.1 and earlier.... published 2024-09-26. https://nvd.nist.gov/vuln/detail/CVE-2024-0132.

[47] NVIDIA Security Bulletin, NVIDIA Container Toolkit.... 2026-06-30; accessed 2026-09-17. https://raw.githubusercontent.com/NVIDIA/product-security/main/2026/5850/5850.md.

[48] gVisor GPU Support (nvproxy): driver ioctls are passed through.... accessed 2026-09-17. https://gvisor.dev/docs/user_guide/gpu/.

[49] CUDA Runtime API, cudaMalloc: 'The memory is not cleared'.... accessed 2026-09-17. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html.

[50] Kata Containers: NVIDIA GPU passthrough via VFIO/IOMMU.... main; accessed 2026-09-17. https://github.com/kata-containers/kata-containers/blob/main/docs/use-cases/NVIDIA-GPU-passthrough-and-Kata.md.

[51] CoreWeave pricing: HGX H100 8-GPU node $49.24 per hour on.... re-read 2026-09-25. https://www.coreweave.com/pricing.

[52] vLLM Supported Models: Qwen3ForCausalLM.... v0.29.0; accessed 2026-09-17. https://docs.vllm.ai/en/latest/models/supported_models.html.

[53] mistralai/Mistral-Small-3.2-24B-Instruct-2506 config.json.... repo created 2025-06-19; accessed 2026-09-17. https://huggingface.co/mistralai/Mistral-Small-3.2-24B-Instruct-2506/raw/main/config.json.

[54] gpt-oss-120b and gpt-oss-20b Model Card (alternating banded and.... 2025-08-08. https://arxiv.org/abs/2508.10925.

[55] Llama 3.1 model card: Llama 3.1 Community License.... release 2024-07-23; accessed 2026-09-17. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md.

[56] Gemma Terms of Use. last modified 2026-04-01; accessed 2026-09-17. https://ai.google.dev/gemma/terms.

[57] Bittensor Docs, Emissions: 0.5 TAO per block after the December.... accessed 2026-09-17 (page shows no date). https://www.bittensor.com/docs/concepts/emissions.

[58] Bittensor Docs, tempo: default 360 blocks; owner range 360 to.... accessed 2026-09-17. https://www.bittensor.com/docs/hyperparameters/tempo.

[59] Bittensor Docs, Registration burn: floating price clamped.... accessed 2026-09-17. https://www.bittensor.com/docs/guides/mining/burn.

[60] subtensor runtime/src/lib.rs: Commitments.set_commitment.... main (spec 462); accessed 2026-09-17. https://github.com/RaoFoundation/subtensor/blob/main/runtime/src/lib.rs.

[61] subtensor pallets/commitments/src/lib.rs: MaxSpace 3,100 bytes.... main; accessed 2026-09-17. https://github.com/RaoFoundation/subtensor/blob/main/pallets/commitments/src/lib.rs.

[62] Bittensor Docs, commit_reveal_weights_enabled: on by default.... accessed 2026-09-17. https://www.bittensor.com/docs/hyperparameters/commit-reveal-weights-enabled.

[63] NVML API Reference: nvmlDeviceGetComputeRunningProcesses.... latest; accessed 2026-09-17. https://docs.nvidia.com/deploy/nvml-api/latest/api/group__nvmlDeviceQueries.html.

[64] BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving.... 2024-01-31. https://arxiv.org/abs/2401.17644.

[65] RULER: What's the Real Context Size of Your Long-Context.... 2024-04-09. https://arxiv.org/abs/2404.06654.

[66] Non-Inferiority Clinical Trials to Establish Effectiveness.... 2016-11. https://www.fda.gov/media/78504/download.

[67] Guideline on the Choice of the Non-Inferiority Margin (CHMP.... 2005-07-27. https://www.ema.europa.eu/en/documents/scientific-guideline/guideline-choice-non-inferiority-margin_en.pdf.

[68] Adding Error Bars to Evals: A Statistical Approach to Language.... 2024-11-01. https://arxiv.org/abs/2411.00640.

[69] Statistical Significance Tests for Machine Translation.... 2004. https://aclanthology.org/W04-3250.pdf.

[70] PyTorch CUDA semantics, memory management: allocator-cached.... stable; accessed 2026-09-17. https://docs.pytorch.org/docs/stable/notes/cuda.html.

[71] GuideLLM Over-Saturation Stopping: significant positive slopes.... v0.7.4; accessed 2026-09-17. https://github.com/vllm-project/guidellm/blob/main/docs/guides/over_saturation_stopping.md.

[72] Always Valid Inference: Bringing Sequential Analysis to A/B.... 2015-12-15 (v3 2019-07-16). https://arxiv.org/abs/1512.04922.

[73] drand Security Model: unpredictable below threshold; unbiasable.... accessed 2026-09-17. https://docs.drand.love/docs/security-model/.

[74] Bittensor Docs, Mining: 256 UID slots; miner emission directed.... accessed 2026-09-17. https://www.bittensor.com/docs/guides/mining.

[75] subtensor v459 run_coinbase.rs 659-802 and subnet_emissions.rs.... commit 70378404b56c; accessed 2026-09-17. https://github.com/opentensor/subtensor.

[76] Bittensor Docs, immunity_period: default 4,096 blocks; pruning.... accessed 2026-09-17. https://www.bittensor.com/docs/hyperparameters/immunity-period.

[77] Finney finalized-block read (own query, script and raw response.... 2026-09-17 15:30:58 UTC. wss://entrypoint-finney.opentensor.ai.

[78] subtensor v459 registration.rs and uids.rs: pruning order among.... commit 70378404b56c; accessed 2026-09-17. https://github.com/opentensor/subtensor.

[79] Bittensor Docs, Registration collateral: optional locked.... accessed 2026-09-17 (references upgrade v437). https://www.bittensor.com/docs/guides/mining/collateral.

[80] subtensor pallets/subtensor dispatches: no slash, ban, or.... main; accessed 2026-09-17. https://github.com/RaoFoundation/subtensor/blob/main/pallets/subtensor/src/macros/dispatches.rs.

[81] quicknet is live on the League of Entropy mainnet (3-second.... 2023-10-16. https://docs.drand.love/blog/2023/10/16/quicknet-is-live/.

[82] EIP-4399: block proposers hold one bit of influence per slot.... created 2021-10-30; status Final. https://eips.ethereum.org/EIPS/eip-4399.

[83] SWC-120: Weak Sources of Randomness from Chain Attributes.... accessed 2026-09-17. https://swcregistry.io/docs/SWC-120/.

[84] Finney RPC subnetInfo_getLockCost at block 9,141,356: subnet.... 2026-09-25. https://entrypoint-finney.opentensor.ai.

[85] CoinGecko simple price API: TAO $299.92 (read 2026-09-25 01:13.... 2026-09-25 (API read). https://api.coingecko.com/api/v3/simple/price?ids=bittensor&vs_currencies=usd.

[86] Bittensor Docs, Running a subnet: registration lock doubles and.... accessed 2026-09-17. https://www.bittensor.com/docs/guides/subnets.

[87] Nebius pricing: HGX H100 $3.85 per GPU-hour on demand, $4.50.... re-read 2026-09-25. https://nebius.com/prices.

[88] subtensor pallets/subtensor/src/coinbase/block_emission.rs: 1.... main; accessed 2026-09-17. https://github.com/RaoFoundation/subtensor/blob/main/pallets/subtensor/src/coinbase/block_emission.rs.

[89] Bittensor Docs, Validating: mainnet stake threshold about 1,000.... accessed 2026-09-17. https://www.bittensor.com/docs/guides/validating.

[90] Finney finalized-block read (own query, retained): block.... 2026-09-17 15:36:18 UTC. wss://entrypoint-finney.opentensor.ai.